

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский ядерный
университет «МИФИ»

Кафедра №42 «Криптология и кибербезопасность»

Отчет о научно-исследовательской работе
по этапу №2

по теме:

РАЗРАБОТКА ИНСТРУМЕНТА ДЛЯ МОНИТОРИНГА ДОСТУПА К
ПАМЯТИ В ЗАДАННОМ ПРОГРАММНОМ КОНТЕКСТЕ В
ОПЕРАЦИОННОЙ СИСТЕМЕ LINUX

Студент:
Группа Б23-505

В. А. Доценко

Научный руководитель:
Кандидат технических наук

И. Ю. Коркин

РЕФЕРАТ

Отчёт 85 с., 16 рис., 4 табл., 19 ист., 1 прил.

МОНИТОРИНГ ПАМЯТИ, LINUX, ДОСТУП К ПАМЯТИ, ЦЕЛОСТНОСТЬ ДАННЫХ, ИНСТРУМЕНТЫ ПРОФИЛИРОВАНИЯ, КОНТЕКСТНЫЙ МОНИТОРИНГ, ЗАЩИТА ПАМЯТИ

Цель работы: создание методики и программного инструмента для контекстного мониторинга и точечной охраны заранее объявленных регионов памяти процесса в Linux, который фиксирует обращения к охраняемому региону вне разрешённых условий и атрибутирует их инициатора по контексту выполнения, включая поток, модуль, инструкцию-инициатор и системный контекст. Задачи исследования включают анализ существующих методов мониторинга и защиты памяти в Linux, проектирование архитектуры инструмента, разработку его прототипа и тестирование на соответствие техническим требованиям.

Объектом исследования является мониторинг доступа к памяти в операционной системе Linux, включая механизмы ОС и аппаратные средства для перехвата и атрибуции операций с выделенными регионами памяти.

Предмет исследования – разработка инструмента, обеспечивающего контекстный мониторинг и точечную охрану произвольных объявленных регионов памяти (как данных, так и фрагментов кода) в Linux, с фиксацией нарушений и атрибуцией их инициатора по потоку, модулю, системному вызову и адресу.

Методологическая основа: Системный подход к анализу механизмов мониторинга памяти и угроз безопасности в операционной системе Linux.

Фокус на конфиденциальности и целостности данных, а также на защите фрагментов кода, как ключевых характеристиках, которые должны быть защищены в процессе мониторинга доступа к памяти.

Сравнительный анализ существующих инструментов мониторинга и технологий безопасности для выявления их преимуществ, недостатков и области применимости.

Подход к разработке нового инструмента, ориентированный на использование штатных средств ядра Linux, применение к целевому приложению без его пересборки и минимизацию воздействия на производительность системы.

Методы исследования: Сравнительный анализ существующих инструментов мониторинга и защиты памяти в Linux и сопредельных подходов, включая нативные средства безопасности операционной системы, механизмы секретной памяти и огораживания, аллокаторы безопасности, а также аппаратные и гипервизорные механизмы.

Моделирование работы системы мониторинга и формирование методики контекстной атрибуции событий доступа к памяти.

Выбор технологического стека на основе сравнительного анализа механизмов перехвата событий доступа к памяти с точки зрения накладных расходов, гранулярности наблюдения и применимости к точечной охране объявленных регионов.

Разработка и тестирование опытного образца модуля перехвата событий доступа к памяти: создание прототипа, фиксирующего обращения к охраняемому региону и восстанавливающего контекст инициатора доступа.

Валидация опытного образца на модельных сценариях доступа к памяти с проверкой корректности срабатывания и полноты атрибуции.

Ожидается, что предложенный инструмент обеспечит сочетание контекстного мониторинга и точечной охраны произвольного региона памяти, включая фрагменты кода, с атрибуцией инициатора доступа, реализуемое штатными средствами ядра Linux без пересборки целевого приложения.

Научная новизна заключается в разработке методики контекстного мониторинга и точечной охраны объявленных регионов памяти с атрибуцией инициатора доступа, распространяющей наблюдение и защиту в том числе на фрагменты кода и реализуемой штатными средствами ядра Linux, что расширяет возможности существующих инструментов и вносит вклад в сферу информационной безопасности.

Результаты работы для этапа №2:

- 1) выбранный технологический стек и методика контекстного мониторинга доступа к памяти;
- 2) опытный образец модуля перехвата событий доступа к памяти;
- 3) результаты валидации опытного образца на модельных сценариях доступа к памяти.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ	9
1 Постановка задачи	12
2 Выбор технологического стека и разработка методики контекстного мониторинга	14
2.1 Конкретизация задачи опытного образца	14
2.1.1 Сценарии использования инструмента	14
2.1.2 Выбор сценария для опытного образца	15
2.2 Обоснование уровня реализации в пользовательском пространстве	17
2.3 Анализ механизмов перехвата и обоснование выбора ptrace	18
2.3.1 Обобщённый примитив контроля доступа к региону памяти	19
2.3.2 Рассматриваемые механизмы	20
2.3.3 Сопоставление механизмов и обоснование выбора	21
2.3.4 Анализ рисков выбранного механизма	22
2.4 Методика контекстного мониторинга доступа к памяти	23
2.5 Алгоритм атрибуции адреса к модулю и функции	27
2.6 Подход к валидации методики	30
2.7 Выводы	31
3 Разработка опытного образца модуля перехвата событий доступа к памяти	34
3.1 Назначение и границы опытного образца	34
3.2 Среда и состав опытного образца	35
3.3 Реализация механизма перехвата	37
3.3.1 Установление контроля над целевым процессом	37
3.3.2 Установка охраны и инъекция вызова mprotect	38
3.3.3 Цикл обработки нарушения и восстановление охраны . . .	41

3.4	Реализация атрибуции	43
3.5	Выявленные технические проблемы и корректировка методики .	46
3.6	Выводы	49
4	Валидация опытного образца на модельных сценариях доступа к памяти	52
4.1	Методика и условия проведения испытаний	52
4.2	Модельные сценарии	54
4.3	Результаты прогона модельных сценариев	57
4.3.1	Результаты запусков сценариев	57
4.3.2	Оценка накладных расходов	61
4.4	Обобщение результатов и границы валидации	62
4.5	Выводы	64
	ЗАКЛЮЧЕНИЕ	66
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	68
	Приложение А Исходный код агента-наблюдателя	70

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчёте о НИР применяются следующие термины с соответствующими определениями:

Агент – внешний по отношению к целевому процессу процесс-наблюдатель, реализующий методику контекстного мониторинга: установку охраны, перехват нарушения, атрибуцию и восстановление охраны.

Атрибуция события – привязка события доступа к элементам исполнения программы и структуры адресного пространства (поток, модуль, функция или инструкция при наличии информации сопоставления адресов).

Инжекция системного вызова – приём, при котором агент временно подменяет состояние регистров целевого процесса так, чтобы тот исполнил требуемый системный вызов в собственном адресном пространстве, после чего исходное состояние регистров восстанавливается.

Инструкция-инициатор – машинная инструкция целевого процесса, исполнение которой вызвало обращение к охраняемой области и, как следствие, нарушение прав доступа; её адрес берётся из указателя инструкций в момент нарушения и служит основой атрибуции.

Контекст выполнения – совокупность сведений о состоянии целевого процесса в момент события доступа: идентификатор потока, состояние регистров, адрес инструкции-инициатора.

Контекстный мониторинг доступа к памяти – наблюдение за обращениями к заранее объявленным областям памяти процесса с привязкой каждого зафиксированного обращения к контексту его выполнения.

Критическая переменная – переменная, от значения которой зависит логика безопасности программы (флаг прохождения аутентификации, указатель на таблицу прав доступа, указатель на функцию) и которая после инициализации не должна изменяться.

Легитимное обращение – обращение к охраняемой области, исходящее из заранее объявленного набора функций, которым доступ к ней разрешён, и потому не подлежащее регистрации как нарушение.

Методика контекстного мониторинга – последовательность действий, образующих метод наблюдения за доступом к охраняемой области, безотносительно деталей программной реализации: символическое объявление области, ограничение прав, отлов нарушения, атрибуция и восстановление охраны.

Модельная программа – вспомогательная программа, содержащая критическую переменную и обращающийся к ней код, используемая как контролируемый испытательный стенд для проверки опытного образца и не входящая в состав инструмента.

Опытный образец – минимально работоспособный прототип модуля перехвата событий доступа к памяти, предназначенный для экспериментальной проверки принятых методических решений.

Охраняемая область – заранее объявленный участок памяти целевого процесса, помещённый под контроль инструмента, обращения к которому вне разрешённых условий подлежат регистрации.

Примитив контроля доступа – обобщённая последовательность из четырёх действий (ограничение прав области, аппаратный отлов нарушения, атрибуция события и восстановление прав), к которой сводятся рассматриваемые механизмы перехвата независимо от различия их интерфейсов.

Символьное объявление области – задание охраняемой области не абсолютным адресом, а через имя соответствующего символа или переменной с последующим разрешением в фактический виртуальный адрес.

Смещение – разность между виртуальным адресом и базовым адресом загрузки модуля, не зависящая от рандомизации размещения адресного пространства и устойчивая между запусками.

Целевой процесс – процесс, за обращениями которого к охраняемой области ведётся наблюдение средствами агента.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете о НИР применяют следующие сокращения и обозначения:

ASLR – Address Space Layout Randomization, рандомизация размещения адресного пространства.

CAP_SYS_PTRACE – capability ядра Linux, дающая мандат на трассировку стороннего процесса средствами ptrace.

DWARF – Debugging With Attributed Record Formats, формат отладочной информации.

ELF – Executable and Linkable Format, формат исполняемых файлов и разделяемых объектов.

PID – Process Identifier, идентификатор процесса.

RIP – Register Instruction Pointer, регистр указателя инструкций в архитектуре x86-64.

TID – Thread Identifier, идентификатор потока.

WP – Write-Protect, защита от записи (режим userfaultfd).

ВВЕДЕНИЕ

Мониторинг доступа к памяти процесса – ключевая задача в области информационной безопасности и разработки программного обеспечения. Эффективное отслеживание операций чтения, записи и исполнения данных в памяти играет важную роль в таких областях, как отладка программ, реверс-инжиниринг, анализ вредоносного ПО, защита от эксплойтов, а также обеспечение целостности данных и выявление внедренного кода. В условиях растущей сложности программ и новых угроз, возникающих из-за уязвимостей, задача мониторинга доступа к памяти становится всё более актуальной.

Актуальность данного исследования обусловлена значительным запросом со стороны государства и растущими потребностями индустрии. Государственная политика в сфере технологического суверенитета и безопасности информационной инфраструктуры, отражённая в Указе Президента РФ № 400 [1], определяет развитие защищённых отечественных технологий как стратегический приоритет. Доктрина информационной безопасности [2] прямо указывает на необходимость создания систем обнаружения и противодействия киберугрозам, включая инструменты низкоуровневого мониторинга. Правовую основу для таких разработок формирует Федеральный закон № 149-ФЗ [3], устанавливающий требования к защите информации от несанкционированного доступа.

Параллельно индустрия демонстрирует устойчивый рост спроса на решения для анализа выполнения кода в реальном времени. Рынок инструментов мониторинга прогнозирует увеличение в сегменте безопасности [4], что связано с распространением атак, направленных на эксплуатацию уязвимостей в памяти. Современные вредоносные программы активно используют техники обфускации и внедрения в память, что делает их недоступными для классических сигнатурных сканеров. Как показывают исследования, анализ содержимого и операций с памятью существенно повышает точность обнаружения скрытых угроз [5,6]. Кроме того, развитие парадигмы конфиденциальных вычислений (Confidential Computing) создаёт новый класс задач: для верификации безопасности изолированных сред выполнения (таких как Intel SGX или AMD SEV) требуются специализированные инструменты, способные контролировать доступ к защищённым областям памяти без полного доверия к операционной

системе [7]. Это формирует потребность в эффективных механизмах, сочетающих низкие накладные расходы с возможностью контекстно-зависимого наблюдения за обращениями к выделенным областям памяти.

Существующие средства, применимые к задаче наблюдения за доступом к памяти, можно разделить на несколько классов, каждый из которых обладает характерными ограничениями. Нативные средства безопасности операционной системы (auditd, SELinux, seccomp) действуют на границе между процессом и ядром: они фиксируют системные вызовы и их аргументы, но не наблюдают обращения к памяти, происходящие между системными вызовами [8]. Механизмы секретной памяти и огораживания (memfd_secret, guard pages [9]) изолируют или скрывают выделенные области, однако реагируют на нарушение лишь аварийным завершением и не атрибутируют его инициатора. Аллокаторы безопасности, среди которых наиболее развит hardened_malloc [10], размещают защитные страницы и карантин вокруг собственных динамических выделений, но не распространяются на произвольные регионы и фрагменты кода и не определяют инициатора нарушения.

Отдельный класс образуют инструменты полной проверки и трассировки памяти. Профайлеры общего назначения, такие как strace и gdb, предоставляют базовые функции отслеживания операций, однако не поддерживают фильтрацию по контексту работы программы. Средства детектирования нарушений memory-safety (AddressSanitizer, Valgrind Memcheck) обеспечивают высокую точность, но требуют пересборки или динамической бинарной инструментализации и приводят к многократному замедлению целевого приложения [11]. Более продвинутые решения, такие как MemGaze, предлагают низкоуровневый анализ трасс доступа к памяти с использованием аппаратной поддержки, что позволяет получать высокое разрешение анализа, но ориентированы на анализ шаблонов доступа и сопряжены с обработкой значительных объёмов данных [11]. Исследования в области аппаратной поддержки, такие как MemTracker [12], предлагают эффективные программируемые механизмы для мониторинга и отладки доступа к памяти, однако требуют модификации аппаратной архитектуры. Подход Blindfold [7] обеспечивает защиту памяти даже при недоверенной операционной системе, но ценой доверенного компонента уровнем выше ядра и модификации последнего. Мониторинг через обработку page faults с использованием механизмов вроде userfaultfd [13] рассматривается как штатное средство ОС, но страдает от высоких накладных расходов при частых обращениях.

Анализ существующих исследований и инструментов выявляет ряд системных недостатков. Одни решения наблюдают системные вызовы, но не видят обращений к памяти между ними (auditd, SELinux); другие проверяют память полно и дорого, ценой пересборки и многократного замедления (AddressSanitizer, Valgrind, MemGaze); третьи точно скрывают или огораживают регион, но без атрибуции инициатора и без защиты кода (memfd_secret, guard pages, hardened_malloc); четвёртые обеспечивают глубокую защиту ценой специализированной аппаратуры или модификации ядра (MemTracker, Blindfold). В результате остаётся непокрытым сочетание свойств: точечная контекстная охрана произвольного объявленного региона памяти, включая фрагменты кода, с атрибуцией инициатора доступа по потоку, модулю и системному контексту, реализуемое штатными средствами ядра Linux и применимое к целевому процессу без его пересборки. Именно этот пробел определяет научную проблему, на решение которой направлено настоящее исследование.

1 Постановка задачи

Целью всей работы является разработать методику и программный инструмент для мониторинга операций чтения/записи/исполнения в память целевого процесса в Linux, с возможностью фильтрации по контексту (поток, модуль, системный вызов, адресный диапазон).

Основные задачи всей работы:

- 1) провести сравнительный анализ существующих подходов к мониторингу доступа к памяти в Linux;
- 2) выбрать технологический стек, разработать методику контекстного мониторинга, реализовать опытный образец модуля перехвата событий доступа к памяти и провести его валидацию на модельных сценариях;
- 3) дополнить разработанный опытный образец механизмами контекстной атрибуции, фильтрации событий по заданной политике и охраны областей данных и кода;
- 4) провести тестирование разработанного прототипа, которое позволит всесторонне оценить работоспособность ключевых функций и убедиться в соответствии продукта техническим требованиям.

В рамках этапа №2 поставлены следующие задачи:

- 1) выбор технологического стека и разработка методики контекстного мониторинга. На основе сравнительного анализа механизмов перехвата событий доступа к памяти, выполненного на предыдущем этапе, определить технологический стек реализации инструмента и разработать методику контекстной охраны объявленных регионов памяти с атрибуцией инициатора доступа;
- 2) разработка опытного образца модуля перехвата событий доступа к памяти. Реализовать прототип модуля, обеспечивающего перехват обращений к охраняемому региону памяти целевого процесса и восстановление контекста инициатора доступа (поток, модуль, инструкция-инициатор, системный контекст);
- 3) валидация опытного образца на модельных сценариях доступа к памяти. Провести экспериментальную проверку прототипа на модельных сценариях обращения к охраняемым регионам с

оценкой корректности срабатывания и полноты восстанавливаемого контекста.

В результате выполнения научно-исследовательской работы на данном этапе предполагается получение следующих результатов:

- 1) отчет по выбору технологий реализации с обоснованием и анализом рисков;
- 2) описания алгоритмов атрибуции адресов к модулям и функциям;
- 3) исходный код опытного образца модуля перехвата событий;
- 4) отчёт о валидации опытного образца с метриками полноты детектирования и накладных расходов.

Ключевая решаемая научная проблема в рамках данной работы заключается в противоречии между высоким уровнем угроз, связанных с эксплуатацией уязвимостей памяти в операционной системе Linux, и недостаточной проработанностью подходов к точечной охране отдельных ценных регионов памяти с установлением инициатора нарушения. С одной стороны, существующие средства защиты памяти либо ограничены перехватом системных вызовов и не наблюдают обращения к памяти между ними, либо обеспечивают полную проверку памяти ценой пересборки приложения и многократного замедления, либо точно скрывают и огораживают регион, но реагируют лишь аварийным завершением и не определяют инициатора доступа. С другой стороны, для распространённых сценариев защиты (охрана криптографического материала в памяти, контроль целостности критических переменных, защита фрагментов кода чувствительных алгоритмов) требуется наблюдение именно на уровне обращений к выделенным регионам с атрибуцией того, какой поток и из какого модуля совершил доступ.

Сохранение указанного противоречия может привести к снижению безопасности систем, использующих Linux, и увеличению рисков эксплуатации уязвимостей памяти. Возможное разрешение данной проблемы заключается в разработке методики и инструмента для точечной контекстной охраны объявленных регионов памяти, реализуемых штатными средствами ядра Linux без пересборки целевого приложения, что обеспечит фиксацию нарушений и атрибуцию их инициатора и тем самым повысит защищённость от несанкционированного доступа к памяти.

2 Выбор технологического стека и разработка методики контекстного мониторинга

Предыдущий этап работы завершился формированием требований к инструменту контекстного мониторинга доступа к памяти и построением концептуальной архитектуры. Вопрос о конкретных технологиях реализации при этом оставался открытым. Настоящая глава решает первую задачу текущего этапа — осуществляет выбор технологического стека и разрабатывает методику контекстного мониторинга, переводя сформулированные ранее требования из плоскости желаемых характеристик в плоскость технических решений. Поскольку реализация всего объёма поставленных задач одновременно нецелесообразна с точки зрения управления рисками, в рамках этапа выделяется один прикладной сценарий и один механизм перехвата событий, доводимые до проработанной методики и опытного образца, тогда как расширение на остальные типы доступа относится к последующим этапам.

2.1 Конкретизация задачи опытного образца

Разрабатываемый инструмент предназначен для охраны заранее объявленных областей памяти процесса и регистрации недопустимых обращений к ним с атрибуцией к контексту выполнения. В наиболее общей постановке под охрану могут помещаться области различной природы, а контролируемым может являться любой из трёх типов доступа — чтение, запись или исполнение. Прежде чем зафиксировать объём опытного образца, целесообразно рассмотреть прикладные сценарии охраны, различающиеся как типом защищаемого объекта, так и семантикой нарушения.

2.1.1 Сценарии использования инструмента

Первый сценарий — охрана секрета в памяти. Под защиту помещается область данных, содержащая криптографический материал: мастер-ключ, приватный ключ или пароль. Большую часть времени область закрыта, а легитимное обращение к ней допускается лишь из ограниченного набора функций, использующих секрет по назначению. Контролируемым типом доступа здесь является чтение: нарушением считается любое обращение к секрету вне разрешённого окна, например со стороны постороннего модуля

или в результате эксплуатации уязвимости, приводящей к чтению чужой области кучи. Этот сценарий развивает задачу защиты обрабатываемой информации в памяти, сформулированную на предыдущем этапе.

Второй сценарий — контроль целостности критической переменной. Под защиту помещается переменная, от значения которой зависит логика безопасности: флаг прохождения аутентификации, указатель на таблицу прав доступа или указатель на функцию. После инициализации такая переменная не должна изменяться, поэтому соответствующая область переводится в режим только для чтения, а контролируемым типом доступа является запись. Нарушением считается любая попытка модификации переменной после её фиксации — характерный признак эксплуатации, при которой переполнение соседнего буфера затирает управляющие данные либо перезаписывается указатель с целью перехвата управления. Данный сценарий развивает задачу контроля целостности глобальных флагов, обозначенную на предыдущем этапе.

Третий сценарий — защита фрагмента кода. Под защиту помещаются страницы, содержащие реализацию чувствительного алгоритма, например криптографической функции. Большую часть времени соответствующая область недоступна, а право на исполнение открывается лишь на время её легитимного вызова. Контролируемыми являются обращения на чтение тела функции (попытка реверс-инжиниринга или извлечения встроенного ключа), на запись в него (инъекция или модификация алгоритма) и на исполнение вне штатного потока вызова. Этот сценарий является наиболее сложным, поскольку требует одновременной работы с несколькими типами доступа и с исполняемыми отображениями.

2.1.2 Выбор сценария для опытного образца

Для опытного образца настоящего этапа из перечисленных сценариев фиксируется второй — контроль целостности критической переменной. Выбор обусловлен несколькими соображениями. Во-первых, этот сценарий обладает наиболее простой семантикой контролируемого доступа: защищаемая область переводится в состояние только для чтения, и единственным наблюдаемым нарушением является запись, тогда как обращения на чтение из той же области остаются легитимными и не требуют обработки. Это исключает необходимость различать допустимые и недопустимые обращения одного типа, что неизбежно

возникает в сценарии охраны секрета. Во-вторых, контроль записи в область, переведённую в режим только для чтения, является прямым продолжением механизма аппаратной проверки прав страниц, рассмотренного на предыдущем этапе, и потому несёт наименьший инженерный риск. В-третьих, данный сценарий не требует работы с исполняемыми отображениями и сопутствующего управления правом на исполнение, что выгодно отличает его от сценария защиты кода.

Сценарии охраны секрета и защиты кода, требующие контроля чтения и исполнения соответственно, выносятся в последующие этапы работы. Такое разделение не нарушает общности предлагаемого подхода: как будет показано далее, все три сценария опираются на единый примитив ограничения прав доступа к области памяти и различаются лишь типом контролируемого доступа и сопутствующей политикой, в силу чего методика, разработанная для контроля записи, естественным образом обобщается на остальные типы доступа.

Наконец, в рамках опытного образца настоящего этапа принимается модельное допущение об однопоточности целевого процесса. Это допущение упрощает методику в части восстановления прав доступа после обработки перехваченного события и позволяет сосредоточиться на корректности базового цикла мониторинга. Корректная обработка многопоточного случая, при которой обращение к охраняемой области может произойти из нескольких потоков одновременно, представляет собой отдельную инженерную задачу и относится к направлениям дальнейшей работы; соответствующие риски рассматриваются в подразделе обоснования выбора механизма перехвата.

Наряду с допущением об однопоточности в опытном образце фиксируется режим его работы. Из двух предусмотренных методикой режимов — запуска целевого приложения под контролем агента и подключения к уже работающему процессу — для опытного образца настоящего этапа выбирается первый. При запуске под контролем агент сам порождает целевой процесс и получает контроль над ним с момента старта, что гарантирует установку охраны до первого обращения к защищаемой области и устраняет необходимость отслеживать состояние уже выполняющегося процесса в момент подключения. Доведение режима подключения к работающему процессу относится к направлениям дальнейшей работы. Кроме того, реакцией опытного образца на нарушение является регистрация события с атрибуцией инициатора, а не пресечение доступа: прототип реализует контекстный

мониторинг и оповещение, тогда как активное пресечение недопустимых обращений рассматривается как возможное развитие инструмента.

2.2 Обоснование уровня реализации в пользовательском пространстве

Прежде чем переходить к сравнению конкретных механизмов перехвата событий доступа к памяти, необходимо определить уровень, на котором инструмент будет реализован. В операционной системе Linux принципиально возможны два подхода: размещение средств наблюдения в пространстве ядра (kernel-space) либо в пользовательском пространстве (user-space). Выбор между ними определяет не только доступный набор технических механизмов, но и характеристики безопасности, переносимости и сопровождаемости инструмента, в силу чего это решение принимается до выбора самого механизма перехвата.

Реализация в пространстве ядра обеспечивает наиболее полный контроль над событиями доступа к памяти, однако сопряжена с принципиальными ограничениями. Первый путь — размещение доверенного компонента на привилегированном уровне выше ядра с одновременной модификацией исходного кода ядра. Подобный подход реализован, в частности, в системе Blindfold, которая защищает память процесса даже от недоверенной операционной системы [14]. Для этого Blindfold вводит небольшой доверенный компонент Guardian, исполняющийся на более высоком уровне привилегий, чем ядро, и требует порядка четырёхсот строк изменений в исходном коде ядра Linux [14]. Такой подход обеспечивает сильные гарантии защиты, но предполагает модификацию ядра и опору на доверенный компонент привилегированного уровня, что противоречит принципу работы на немодифицированном (стоковом) ядре.

Второй путь реализации в пространстве ядра — оформление инструмента в виде загружаемого модуля ядра. Этот вариант не требует пересборки ядра, однако обладает собственным существенным недостатком с точки зрения безопасности. Код, исполняющийся в пространстве ядра, входит в доверенную вычислительную базу системы и обладает неограниченным доступом к её ресурсам, вследствие чего любой дефект в таком модуле становится уязвимостью уровня ядра и расширяет поверхность атаки всей системы. Это прямо противоречит сформулированному на предыдущем этапе

эксплуатационному требованию, согласно которому инструмент не должен нарушать целостность системы и создавать в ней уязвимостей. Поскольку инструмент позиционируется как средство обеспечения безопасности, привнесение им самим дополнительных рисков уровня ядра следует считать неприемлемым.

С учётом изложенного в качестве уровня реализации выбирается пользовательское пространство при условии работы на немодифицированном ядре с опорой на штатные интерфейсы операционной системы. Такой выбор исключает расширение доверенной вычислительной базы, сохраняет переносимость инструмента между совместимыми версиями ядра и соответствует требованию невмешательства в целостность системы. Платой за это является ограниченность доступного инструментария теми механизмами наблюдения, которые ядро штатно предоставляет пользовательским процессам, — однако, как будет показано далее, имеющегося набора интерфейсов достаточно для реализации требуемой функциональности.

Выбор пользовательского пространства влечёт за собой важное следствие, сужающее круг рассматриваемых механизмов. Согласно концепции инструмента, охрана и наблюдение должны вестись из агента, отделённого от целевого процесса: средство контроля, исполняющееся внутри наблюдаемого процесса, разделяет с ним адресное пространство и привилегии, а потому может быть отключено в результате компрометации этого процесса столь же легко, как и его собственный код обработки исключений. Внешний по отношению к цели агент такой возможности атакующему не предоставляет. Поэтому из числа пригодных механизмов исключаются те, что предполагают исполнение кода наблюдения непосредственно в контексте целевого процесса, и предпочтение отдаётся механизмам, допускающим вынесение логики перехвата и атрибуции во внешний процесс. Это требование становится одним из ключевых критериев при последующем сравнении конкретных технологий перехвата.

2.3 Анализ механизмов перехвата и обоснование выбора ptrace

Определив пользовательское пространство как уровень реализации, перейдём к выбору конкретного механизма перехвата событий доступа к охраняемой области. Как было отмечено ранее, выбор механизма не ограничивается потребностями текущего этапа: поскольку последующее

наращивание функциональности выполняется поверх перехвата, именно он определяет предельные возможности инструмента на всех дальнейших этапах. В частности, ключевая для концепции инструмента атрибуция события вплоть до инициировавшей его инструкции возможна лишь в том случае, если механизм перехвата предоставляет состояние выполнения в момент нарушения. Поэтому критерий выбора включает не только достаточность для решения текущей задачи контроля записи, но и способность механизма обеспечить требуемую глубину атрибуции и работу из внешнего по отношению к цели агента.

2.3.1 Обобщённый примитив контроля доступа к региону памяти

Несмотря на различие конкретных интерфейсов, рассматриваемые механизмы сводятся к единому примитиву контроля доступа, состоящему из четырёх последовательных действий:

- 1) ограничение прав доступа к охраняемой области таким образом, чтобы контролируемый тип доступа (в рамках выбранного сценария — запись) стал невозможен без срабатывания аппаратной проверки прав;
- 2) аппаратный отлов нарушения, при котором попытка доступа к области с недостаточными правами порождает синхронное исключение, передаваемое механизму наблюдения;
- 3) атрибуция перехваченного события, то есть привязка его к контексту выполнения — потоку, модулю и, при наличии необходимых сведений, инициировавшей инструкции;
- 4) восстановление прав и продолжение выполнения, обеспечивающее как корректное завершение легитимной операции, так и сохранение охраны для последующих обращений.

Разрабатываемый инструмент опирается на тот же принцип ограничения прав и отлова нарушения, однако отличается от перечисленных решений в двух существенных отношениях. Во-первых, реакцией на нарушение является не аварийное завершение и не молчаливое снятие защиты, а наблюдение с атрибуцией нарушителя: инструмент фиксирует, какой поток, из какого модуля и по какому адресу совершил недопустимое обращение. Во-вторых, под охрану помещается произвольная объявленная область, а не только динамические выделения собственного аллокатора, как в случае защитных страниц

вокруг heap-аллокаций. Именно сочетание контроля произвольной области с атрибуцией инициатора отличает предлагаемый подход от существующих механизмов охранных страниц.

2.3.2 Рассматриваемые механизмы

Дальнейшее сравнение проводится по четырём осям: поддерживаемые типы контролируемого доступа (чтение, запись, исполнение), возможность вынесения наблюдения во внешний процесс (внешность), достижимая глубина атрибуции и профиль накладных расходов.

Механизм `userfaultfd` в режиме защиты от записи (`write-protect`) позволяет внешнему процессу-обработчику регистрировать страницы целевого адресного пространства и получать уведомления о попытках записи в них, приостанавливая нарушивший поток до разрешения события. Этот механизм является внешним и обладает умеренными накладными расходами, а сообщаемая информация включает идентификатор нарушившего потока. Однако предоставляемого контекста недостаточно для атрибуции на уровне инструкции: механизм сообщает адрес нарушения и поток, но не передаёт состояние регистров и, в частности, адрес инструкции-инициатора. Кроме того, в базовом виде он ориентирован преимущественно на анонимную память и на контроль записи, что сужает его применимость для последующего расширения на чтение и исполнение.

Связка `mprotect` с обработчиком сигнала `SIGSEGV` основана на снятии прав доступа с охраняемой области средствами `mprotect` и перехвате возникающего при нарушении сигнала `SIGSEGV`. Этот подход поддерживает контроль всех трёх типов доступа и предоставляет обработчику полный контекст выполнения через структуру состояния потока, включая адрес инструкции-инициатора. Вместе с тем обработчик сигнала выполняется непосредственно в контексте целевого процесса, разделяя с ним адресное пространство и привилегии. Это противоречит требованию внешности: код наблюдения, размещённый внутри цели, может быть отключён или подменён в результате компрометации процесса столь же легко, как и любая иная его часть, что делает данный механизм непригодным в качестве основы инструмента безопасности.

Связка `ptrace` с `mprotect` использует тот же приём ограничения прав, однако перехват возникающего нарушения выполняется внешним процессом-

агентом. Агент подключается к цели средствами ptrace, снимает права с охраняемой области, а при нарушении получает управление через ожидание изменения состояния отслеживаемого процесса, после чего считывает состояние его регистров. Тем самым механизм одновременно удовлетворяет требованию внешности и обеспечивает атрибуцию на уровне инструкции, поскольку предоставляет агенту адрес инструкции-инициатора, состояние стека и идентификатор потока. Платой за это являются повышенные накладные расходы, обусловленные переключениями контекста между агентом и целью, а также необходимость пошагового исполнения для восстановления охраны после легитимного доступа.

Механизмы изоляции на основе охранных страниц, включая интерфейс MADV_GUARD_INSTALL, переводят область в состояние полной недоступности и реагируют на обращение к ней аварийным завершением или ошибкой без сбора контекста нарушителя [15]. Поскольку такие механизмы не предусматривают ни различия типа доступа, ни атрибуции инициатора, они не отвечают задаче инструмента и далее не рассматриваются.

2.3.3 Сопоставление механизмов и обоснование выбора

Сведём результаты сравнения рассмотренных механизмов в таблицу, сопоставив их по выделенным осям и по соответствию ключевым требованиям к инструменту.

Таблица 1 — Сравнение механизмов перехвата событий доступа к памяти

Механизм	Типы доступа	Внешность	Атрибуция	Накладные расходы
userfaultfd (WP)	W	да	до потока	умеренные
mprotect + SIGSEGV	R/W/X	нет	до инструкции	низкие
ptrace + mprotect	R/W/X	да	до инструкции	высокие
Охранные страницы	нет (крах)	—	нет	низкие

Сопоставление показывает, что ни userfaultfd, ни охранные страницы не обеспечивают атрибуции на уровне инструкции, а связка mprotect с SIGSEGV, предоставляя такую атрибуцию, не удовлетворяет требованию внешности.

Единственным механизмом, совмещающим внешность, поддержку всех типов доступа и атрибуцию вплоть до инициировавшей инструкции, является связка ptrace с mprotect. Поскольку именно глубина атрибуции и внешность определяют пригодность механизма как основы для последующих этапов работы, в качестве базового механизма перехвата выбирается связка ptrace с mprotect. Контроль записи, требуемый выбранным сценарием, реализуется в её рамках непосредственно, а потенциал расширения на чтение и исполнение сохраняется без смены базового механизма.

2.3.4 Анализ рисков выбранного механизма

Выбор связки ptrace с mprotect сопряжён с рядом рисков, которые необходимо учитывать при разработке опытного образца:

- 1) накладные расходы. Каждое срабатывание влечёт несколько переключений контекста между агентом и целью, поэтому при высокой частоте обращений к охраняемой области производительность снижается; риск смягчается избирательностью охраны — под защиту помещается небольшое число редко используемых областей, а не вся память процесса;
- 2) сложность пошагового исполнения. Восстановление охраны после легитимного доступа требует временного возврата прав, исполнения одной инструкции в пошаговом режиме и повторного снятия прав, что усложняет логику агента и требует аккуратной реализации;
- 3) требование привилегий. Подключение к чужому процессу средствами ptrace ограничено политиками безопасности (в частности, параметром ptrace_score и мандатом CAP_SYS_PTRACE), вследствие чего инструмент в общем случае требует повышенных привилегий, что согласуется со сформулированным ранее эксплуатационным требованием;
- 4) потеря охраны при переотображении области. Изменение карты виртуальной памяти целевого процесса (например, переотображение или освобождение области) способно сбросить установленные ограничения прав, поэтому агент должен отслеживать подобные изменения и переустанавливать охрану;
- 5) многопоточная гонка при пошаговом исполнении. В интервале между восстановлением прав и повторным их снятием обращение

к охраняемой области может произойти из другого потока в обход контроля. Корректная обработка этого случая представляет собой отдельную задачу и относится к направлениям дальнейшей работы; в рамках опытного образца настоящего этапа она снимается принятым допущением об однопоточности целевого процесса.

Перечисленные риски не носят запретительного характера и либо смягчаются избирательностью охраны и принятыми допущениями, либо относятся к последующим этапам, не затрагивая корректность базового цикла мониторинга, разрабатываемого на текущем этапе.

2.4 Методика контекстного мониторинга доступа к памяти

На основе выбранного механизма перехвата сформулируем методику контекстного мониторинга доступа к памяти. Под методикой здесь понимается последовательность действий, образующих метод наблюдения, безотносительно деталей его программной реализации. Методика опирается на введённый ранее обобщённый примитив контроля доступа — ограничение прав области, отлов нарушения, атрибуция события и восстановление прав — и конкретизирует его применительно к выбранной связке ptrace и mprotect и к выбранному сценарию контроля целостности критической переменной. Изложение ведётся в предположении, что под охрану помещается заранее объявленная область данных, которая после инициализации не должна изменяться, а контролируемым типом доступа является запись.

Метод состоит из следующих последовательных шагов:

- 1) символьное объявление охраняемой области. Область задаётся не абсолютным адресом, а символически — через имя соответствующего символа или переменной. Это обусловлено тем, что фактический виртуальный адрес области меняется от запуска к запуску вследствие рандомизации адресного пространства (ASLR) и в общем случае заранее неизвестен ни пользователю, ни инструменту;
- 2) установление наблюдения за целевым процессом средствами ptrace. Методика допускает два режима: запуск целевого приложения под контролем агента и подключение к уже выполняющемуся процессу;
- 3) разрешение символа в текущий виртуальный адрес. Агент сопоставляет объявленный символ с фактическим адресом в

адресном пространстве цели, используя информацию о загруженных модулях и их символах;

- 4) ограничение прав доступа к области. Агент инициирует в адресном пространстве цели снятие права записи с охраняемой области, переводя соответствующие страницы в состояние, доступное только для чтения, в результате чего любая последующая попытка записи приводит к аппаратному нарушению;
- 5) исполнение целевого процесса до возникновения нарушения. Пока обращений на запись к охраняемой области не происходит, цель исполняется штатно и без вмешательства;
- 6) перехват нарушения агентом. Возникающее при попытке записи синхронное исключение передаётся ядром агенту, который получает управление извне через ожидание смены состояния отслеживаемого процесса, не полагаясь на какой-либо код обработки внутри самой цели;
- 7) чтение контекста выполнения. Агент считывает состояние потока цели в момент нарушения — адрес инструкции-инициатора, регистры и идентификатор потока, — которое служит исходными данными для последующей атрибуции;
- 8) формирование записи о событии, включающей адрес обращения, тип доступа и сведения о контексте выполнения;
- 9) восстановление прав и одиночный шаг. Агент временно возвращает охраняемой области право записи, исполняет одну инструкцию цели в пошаговом режиме (тем самым давая нарушившей инструкции завершиться) и затем повторно снимает право записи, восстанавливая охрану;
- 10) продолжение исполнения. Цель возобновляет работу, а метод возвращается к шагу ожидания следующего нарушения; цикл повторяется до останова мониторинга.

Несколько аспектов метода требуют отдельного пояснения. Первый касается различия легитимного и недопустимого доступа. В рамках выбранного сценария и текущего этапа принята политика «только чтение после инициализации»: охраняемая область фиксируется после её заполнения, и любое последующее обращение на запись трактуется как нарушение. Более тонкое различие, при котором часть записей признаётся легитимной

в зависимости от контекста инициатора, опирается на ту же атрибуцию и закладывается как направление развития, не требующее изменения базового метода.

Второй аспект связан с одноразовостью срабатывания, характерной для механизмов на основе ограничения прав. Простой переход прав даёт лишь однократный сигнал: после первого же нарушения для продолжения исполнения право доступа приходится вернуть, вследствие чего охрана снимается. По этой причине механизм `soft-dirty` в Linux, отслеживающий факт записи в страницу, также фиксирует лишь первое обращение в пределах интервала наблюдения [16]. Именно поэтому метод включает повторное снятие права записи после одиночного шага: восстановление охраны на каждом цикле обеспечивает непрерывное, а не однократное детектирование обращений к области.

Третий аспект — модель параллелизма целевого процесса. Методика в изложенном виде предполагает однопоточную модель цели. Это допущение существенно на шаге восстановления прав: в интервале между возвратом права записи и его повторным снятием другой поток мог бы обратиться к охраняемой области в обход контроля. Корректная обработка такого случая представляет собой отдельную задачу синхронизации и относится к направлениям дальнейшей работы; в рамках опытного образца настоящего этапа она снимается принятым допущением об однопоточности.

Описанную последовательность действий удобно представить в виде единого цикла работы метода. На рисунке приведена концептуальная блок-схема, отражающая как однократные подготовительные шаги (объявление области, подключение, разрешение символа и первичное ограничение прав), так и повторяющийся цикл наблюдения (исполнение, отлов нарушения, атрибуция, формирование записи и восстановление охраны).

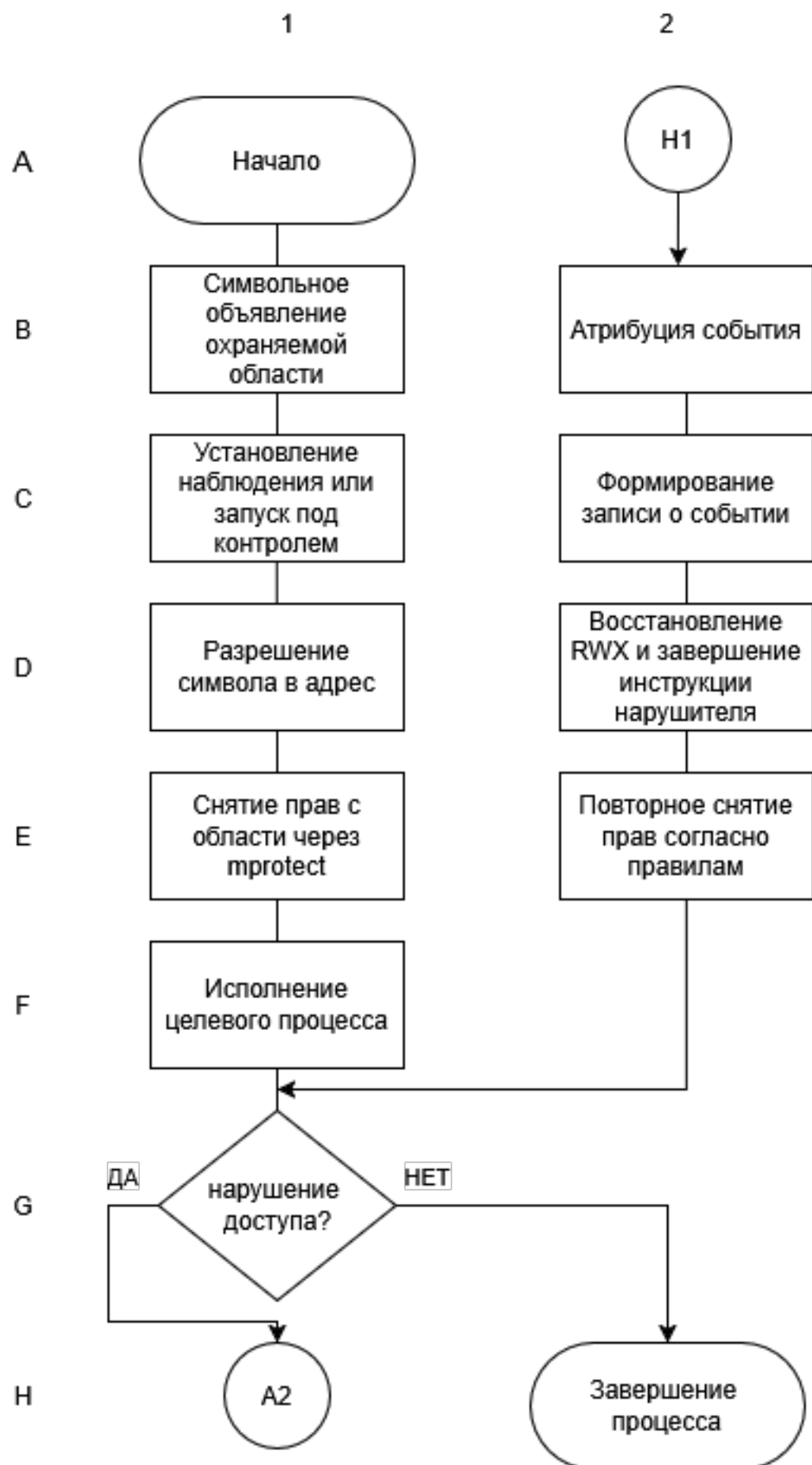


Рисунок 1 — Блок-схема цикла методики контекстного мониторинга

Представленная схема описывает метод на концептуальном уровне и сознательно не раскрывает внутреннее устройство отдельных шагов. В частности, шаг атрибуции изображён как единый блок: его развёртывание в последовательность действий по сопоставлению адреса с модулем и функцией рассматривается в следующем подразделе. Аналогичным образом детальная, реализационная блок-схема агента — с конкретными вызовами и обработкой их результатов — приводится в главе, посвящённой разработке опытного образца, поскольку относится уже к воплощению метода, а не к его постановке.

2.5 Алгоритм атрибуции адреса к модулю и функции

Перехваченное событие доступа в исходном виде представляет собой набор низкоуровневых данных: адрес обращения к охраняемой области и состояние потока в момент нарушения. Сами по себе эти данные малоинформативны для пользователя, поскольку не сообщают, какой компонент программы инициировал обращение. Задача атрибуции состоит в том, чтобы превратить сырое событие в осмысленную привязку к элементам исполнения программы — потоку, модулю и, при наличии необходимых сведений, функции. Настоящий подраздел раскрывает блок атрибуции, изображённый единым элементом на блок-схеме метода в предыдущем подразделе, и описывает соответствующий алгоритм на концептуальном уровне, не затрагивая деталей его программной реализации.

Входными данными алгоритма служат сведения, доступные из перехваченного контекста: адрес обращения к охраняемой области, вызвавший нарушение, и адрес инструкции-инициатора, то есть значение указателя инструкций потока в момент нарушения. Первый отвечает на вопрос, к какой именно области произошло обращение, второй — на вопрос, какой участок кода это обращение совершил. Основная содержательная работа атрибуции выполняется именно над адресом инструкции-инициатора, поскольку он позволяет локализовать обращение в коде программы.

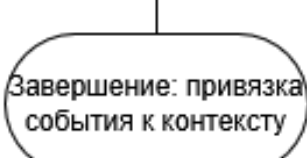
Алгоритм атрибуции состоит из следующих шагов:

- 1) чтение карты отображений целевого процесса. Из интерфейса `/proc/<pid>/maps` извлекается перечень областей виртуального адресного пространства цели с указанием границ каждой области и связанного с ней источника — исполняемого файла, разделяемой библиотеки или анонимной памяти [17];

- 2) поиск отображения, в границы которого попадает адрес инструкции-инициатора. Среди полученных областей определяется та, чей диапазон адресов содержит искомый адрес;
- 3) определение модуля и смещения. По найденному отображению устанавливается модуль (исполняемый файл или конкретная библиотека), а вычитанием базового адреса отображения из адреса инструкции-инициатора получается смещение внутри модуля, не зависящее от рандомизации адресного пространства;
- 4) разрешение смещения в имя функции. При наличии в модуле таблицы символов формата ELF [18] или отладочной информации формата DWARF [19] полученное смещение сопоставляется с именем функции, в границы которой оно попадает;
- 5) определение потока. Идентификатор потока, совершившего обращение, берётся из контекста перехваченного события и присоединяется к результату атрибуции.

Результатом работы алгоритма является привязка события к контексту выполнения, достижимая глубина которой зависит от доступной о модуле информации. Если модуль содержит символьную или отладочную информацию, атрибуция доводится до имени функции; если такая информация отсутствует (например, у модуля, собранного без таблицы символов), атрибуция ограничивается модулем и смещением внутри него. Важно подчеркнуть, что эта граница точности является принципиальным свойством метода, а не дефектом конкретной реализации: восстановить имя функции в отсутствие символьной информации невозможно, однако привязка к модулю и смещению остаётся определённой в любом случае и сама по себе достаточна для локализации источника обращения с точностью до компонента программы.

Описанную последовательность действий иллюстрирует приведённая ниже блок-схема алгоритма атрибуции.



29

Схема отражает ключевую развилку алгоритма: после определения модуля и смещения дальнейшая детализация зависит от наличия символьной или отладочной информации, вследствие чего выходом алгоритма является либо привязка к функции, либо привязка к модулю и смещению. Данная блок-схема представляет собой один из заявленных результатов настоящего этапа работы. В отличие от блок-схемы предыдущего подраздела, описывающей цикл метода в целом, настоящая схема раскрывает внутреннее устройство единственного его шага — атрибуции — и не затрагивает вопросов перехвата и управления исполнением цели, отнесённых к методике.

2.6 Подход к валидации методики

Разработанная методика нуждается в проверке на способность обнаруживать недопустимые обращения к охраняемой области и корректно их атрибутировать. Настоящий подраздел в общих чертах определяет подход к такой проверке — модель и перечень сценариев, — тогда как детальное описание испытаний и их результаты относятся к отдельной главе, посвящённой валидации опытного образца.

В основу подхода положена модель, в которой охраняемая программа содержит критическую переменную, объявленную ценной областью, а некоторый участок кода совершает недопустимое обращение к этой области, которое инструмент должен перехватить и атрибутировать. В соответствии с принятым допущением об однопоточности недопустимое обращение моделируется не отдельным параллельным потоком, а кодом в составе той же однопоточной программы — посторонним модулем или функцией, либо результатом ошибки управления памятью.

Перечень модельных сценариев включает:

- 1) легитимную запись из ожидаемого контекста — обращение к области из участка кода, которому разрешено её изменять до фиксации области как неизменяемой; сценарий проверяет отсутствие ложных срабатываний на законном пути исполнения;
- 2) запись из постороннего модуля — обращение на запись из модуля, которому изменять область не положено; сценарий проверяет корректность отлова и атрибуцию на уровне модуля-инициатора;
- 3) запись из посторонней функции того же модуля — обращение из участка кода, отличного от легитимного, в пределах одного модуля;

сценарий проверяет способность атрибуции различать инициатора на уровне функции;

- 4) запись вследствие переполнения соседнего буфера — модификацию охраняемой области в результате выхода записи за границы смежного буфера; сценарий воспроизводит типовой механизм эксплуатации;
- 5) контрольное обращение к незащищённой области — обращение к области, не помещённой под охрану; сценарий проверяет избирательность контроля, при которой подобное обращение не должно порождать событий.

В совокупности перечисленные сценарии охватывают три проверяемых свойства методики: корректность отлова недопустимых обращений, отсутствие ложных срабатываний на легитимном пути и базовую атрибуцию инициатора. В пределах принятых допущений ожидаемая полнота детектирования является полной: всякое недопустимое обращение на запись к охраняемой области, совершённое в рамках однопоточной модели, подлежит перехвату.

За границами принятых допущений остаётся проверка обращений из параллельных потоков, требующая обработки многопоточного случая на шаге восстановления прав. Соответствующий сценарий — запись в охраняемую область из стороннего потока — относится к направлениям дальнейшей работы и в перечень валидации настоящего этапа не включается.

2.7 Выводы

В настоящей главе решена первая задача этапа — осуществлён выбор технологического стека и разработана методика контекстного мониторинга доступа к памяти. Изложение велось от конкретизации задачи к обоснованию технических решений и далее к методике и плану её проверки, в результате чего сформирован завершённый набор проектных решений, на котором основывается разработка опытного образца.

Основные результаты главы состоят в следующем:

- 1) конкретизирована задача опытного образца. Из трёх прикладных сценариев охраны — секрета в памяти, целостности критической переменной и фрагмента кода — для реализации на текущем этапе выбран контроль целостности критической переменной как обладающий наиболее простой семантикой контролируемого доступа

и наименьшим инженерным риском; зафиксированы модельные допущения опытного образца: однопоточность целевого процесса, режим запуска цели под контролем агента и реакция в виде регистрации события с атрибуцией без пресечения доступа;

- 2) обоснован уровень реализации. Показано, что размещение средств наблюдения в пространстве ядра — как путём модификации ядра с доверенным компонентом привилегированного уровня, так и в виде загружаемого модуля — противоречит принципу работы на немодифицированном ядре и требованию невмешательства в целостность системы; в качестве уровня реализации выбрано пользовательское пространство, что влечёт требование вынесения логики наблюдения во внешний по отношению к цели агент;
- 3) выбран механизм перехвата. По итогам сравнения механизмов `userfaultfd`, связки `mprotect` с обработчиком `SIGSEGV`, связки `ptrace` с `mprotect` и механизмов охранных страниц по осям типа доступа, внешности, глубины атрибуции и накладных расходов в качестве базового механизма выбрана связка `ptrace` с `mprotect` — единственная, совмещающая внешность, поддержку всех типов доступа и атрибуцию вплоть до инициировавшей инструкции; проведён анализ сопутствующих рисков и указаны меры их смягчения;
- 4) разработана методика контекстного мониторинга. Метод представлен как последовательность шагов от символьного объявления охраняемой области до восстановления охраны после перехвата нарушения и оформлен в виде концептуальной блок-схемы цикла работы; отдельно разобран алгоритм атрибуции адреса к модулю и функции с указанием границ достижимой точности и представлен соответствующей блок-схемой;
- 5) определён подход к валидации методики. Сформирован перечень модельных сценариев доступа к охраняемой области с ожидаемым исходом детектирования, дающий оценку ожидаемой полноты обнаружения нарушений в пределах принятых допущений.

Полученные решения согласованы между собой и подчинены единому критерию — пригодности выбранного механизма не только для текущей задачи контроля записи, но и для последующего расширения на контроль чтения и исполнения без смены технологической основы. Тем самым сформированная

методика и обоснованный выбор технологического стека образуют основу для решения следующей задачи этапа — разработки опытного образца модуля перехвата событий доступа к памяти.

3 Разработка опытного образца модуля перехвата событий доступа к памяти

В предыдущей главе были зафиксированы технологический стек — реализация в пользовательском пространстве на основе связки ptrace и mprotect — и методика контекстного мониторинга доступа к памяти. Настоящая глава решает вторую задачу этапа — разработку опытного образца модуля перехвата событий доступа к памяти. Разрабатываемый образец не является полнофункциональной реализацией инструмента: это минимально работоспособный прототип, предназначенный для экспериментальной проверки гипотез о работоспособности выбранных механизмов и связанных с ними накладных расходах перед переходом к последующему этапу. В состав задачи входят реализация перехвата нарушений доступа к охраняемой области, реализация атрибуции перехваченного события к контексту выполнения, документирование выявленных в ходе разработки технических проблем и, при необходимости, уточнение методики по результатам её практического воплощения. Количественная проверка образца на модельных сценариях доступа к памяти выходит за рамки настоящей главы и составляет предмет следующей.

3.1 Назначение и границы опытного образца

Опытный образец, разрабатываемый в настоящей главе, представляет собой не законченный программный продукт, а средство экспериментальной проверки решений, принятых на этапе разработки методики. Его назначение — подтвердить принципиальную работоспособность выбранной связки ptrace и mprotect в составе единого цикла мониторинга. Критерием успеха образца считается не полнота функциональности и не достижение целевых показателей производительности, а демонстрация того, что заложенный в методику цикл — ограничение прав, отлов нарушения, атрибуция и восстановление охраны — реализуем на немодифицированном ядре и приводит к ожидаемому поведению на контролируемых сценариях. Полученные при этом наблюдения служат основанием для уточнения методики и планирования последующих этапов работы.

Образец предназначен для проверки нескольких предположений, на которых основана разработанная методика. Во-первых, проверяется реализуемость внешнего перехвата записи в охраняемую область на

немодифицированном ядре, то есть возможность агента, отделённого от целевого процесса, обнаруживать нарушения прав доступа без внедрения кода в саму цель. Во-вторых, проверяется достижимость атрибуции перехваченного события вплоть до инициировавшей его инструкции при опоре на штатно доступный контекст выполнения. Кроме того, образец сохраняет принцип избирательной охраны, при котором под защиту помещается небольшое число редко используемых областей, а не всё адресное пространство процесса; тем самым в самой конструкции образца заложено ограничение источника накладных расходов, количественная характеристика которого относится к последующей проверке образца.

Образец наследует модельные допущения, зафиксированные в предыдущей главе: однопоточность целевого процесса, режим запуска цели под контролем агента, контроль единственного типа доступа — записи — и реакцию на нарушение в виде регистрации события с атрибуцией инициатора без пресечения доступа. Эти допущения определяют объём реализуемой в образце функциональности и согласованы с выбранным прикладным сценарием контроля целостности критической переменной.

За границами образца сознательно оставлены режим подключения к уже работающему процессу, контроль чтения и исполнения, обработка многопоточного случая и контекстная фильтрация событий. Их исключение не препятствует проверке перечисленных предположений: каждое из них относится либо к расширению на другие типы доступа и режимы работы, либо к функциональности, надстраиваемой над базовым циклом перехвата, и потому не затрагивает корректность самого цикла, проверяемого на текущем этапе. Тем самым сокращённый объём образца достаточен для вынесения суждения о работоспособности выбранных механизмов и не обесценивает результатов проверки.

3.2 Среда и состав опытного образца

Опытный образец реализуется со следующими характеристиками среды и способа сборки:

- 1) аппаратная платформа — x86-64; выбор обусловлен распространённостью архитектуры и тем, что именно для неё в предыдущей главе рассматривались аппаратная проверка прав страниц и формат диагностического контекста нарушения;

- 2) операционная система — Linux с немодифицированным ядром, в соответствии с принятым принципом работы на немодифицированном ядре;
- 3) стандартная библиотека — glibc;
- 4) язык реализации — C;
- 5) способ сборки — штатный системный компилятор без флагов инструментализации, что согласуется с требованием работы с целевыми приложениями в их обычных сборках.

Выбор языка C определяется характером задачи. Основная часть кода образца представляет собой непосредственное обращение к системным интерфейсам ядра и стандартной библиотеки — управление трассируемым процессом, чтение его памяти и регистров, разбор карты отображений и символьной информации, — для которых язык C обеспечивает прямое соответствие коду системных вызовов без промежуточных абстракций. Применение объектно-ориентированных средств не даёт здесь существенного выигрыша, поскольку логика образца представляет собой машину состояний над трассируемым процессом, а не иерархию объектов. Вместе с тем минимизация не прямых абстракций между кодом образца и ядром повышает достоверность первичной оценки накладных расходов, не позволяя отнести к свойствам механизма издержки, привнесённые средствами языка.

Образец состоит из двух компонентов:

- 1) агент-наблюдатель, реализующий разработанную методику: он порождает целевой процесс под своим контролем, ограничивает права доступа к охраняемой области, перехватывает возникающее при нарушении исключение, считывает контекст выполнения и выполняет атрибуцию события. Агент является собственно воплощением метода и представляет основной интерес с точки зрения проверяемых предположений;
- 2) модельная защищаемая программа, выступающая испытательным стендом: она содержит критическую переменную, объявляемую охраняемой областью, и участки кода, обращающиеся к этой переменной как допустимым, так и недопустимым образом.

Модельная программа не входит в состав разрабатываемого инструмента: её единственное назначение — предоставить контролируемые условия для проверки заложенных в методику предположений. Размещение

критической переменной и характер обращающегося к ней кода в такой программе полностью управляемы, что позволяет воспроизводимо порождать как легитимные, так и недопустимые обращения и тем самым проверять корректность отлова, атрибуции и отсутствие ложных срабатываний. В реальном применении роль защищаемой программы исполняет произвольное целевое приложение, к которому агент не предъявляет специальных требований; модельная же программа используется исключительно как инструмент экспериментальной проверки на текущем этапе.

3.3 Реализация механизма перехвата

Настоящий подраздел решает первую из задач этапа — реализацию механизма перехвата событий доступа к памяти — и описывает, каким образом последовательность шагов методики, изложенная в предыдущей главе на уровне метода, воплощается в конкретные примитивы операционной системы. Если методика оперирует понятиями ограничения прав, отлова нарушения и восстановления охраны безотносительно их программного выражения, то опытный образец сопоставляет каждому из этих понятий определённый системный вызов или их последовательность. Изложение организовано по трём составляющим механизма: установление контроля над целевым процессом, установка охраны над объявленной областью и обработка возникающего нарушения с последующим восстановлением охраны. Такое деление соответствует разделению однократных подготовительных действий и повторяющегося цикла наблюдения, введённому в методике. Реализация выполнена на языке C для архитектуры x86-64 в среде Linux; выбор примитивов и порядок их применения определяются принятыми ранее проектными решениями, прежде всего требованием вынесения логики наблюдения во внешний по отношению к цели процесс-агент.

3.3.1 Установление контроля над целевым процессом

Установление контроля над целевым процессом реализует тот шаг методики, на котором агент берёт наблюдаемый процесс под управление средствами ptrace. В соответствии с зафиксированным для опытного образца режимом запуска цели под контролем агента контроль устанавливается с момента порождения процесса. Порождение трассируемой цели выполняется последовательностью из трёх действий:

- 1) вызов `fork`, порождающий дочерний процесс;
- 2) операция `ptrace` с запросом `PTRACE_TRACEME` в дочернем процессе, объявляющая его трассируемым;
- 3) вызов `execlve`, замещающий образ дочернего процесса образом целевой программы.

Такая последовательность гарантирует, что к моменту начала исполнения целевого кода процесс уже находится под наблюдением, а первый же системный вызов `execlve` приводит к его остановке с сигналом `SIGTRAP`, предоставляя агенту возможность выполнить подготовительные действия до первого обращения к защищаемой области. Это устраняет необходимость отслеживать состояние уже выполняющегося процесса, характерную для режима подключения, отнесённого к направлениям дальнейшей работы.

После установления контроля агент входит в главный цикл наблюдения, построенный вокруг ожидания изменения состояния отслеживаемого процесса. Агент вызывает `waitpid`, который блокирует выполнение агента до очередной остановки цели, и по возвращении из вызова классифицирует причину остановки, различая три случая:

- 1) завершение процесса, прекращающее цикл наблюдения;
- 2) аварийное прекращение процесса по сигналу;
- 3) остановку с доставкой сигнала, несущую основную содержательную нагрузку, поскольку перехватываемое нарушение прав доступа проявляется именно как остановка с сигналом `SIGSEGV`.

Остановки, вызванные иными причинами, обрабатываются отдельно: служебные остановки, не связанные с нарушением охраняемой области, не порождают событий и сопровождаются продолжением исполнения. Тем самым главный цикл агента отделяет события доступа к памяти, подлежащие регистрации и атрибуции, от прочих изменений состояния процесса.

3.3.2 Установка охраны и инъекция вызова `mprotect`

Установка охраны над объявленной областью представляет собой ключевой инженерный узел опытного образца, поскольку именно здесь проявляется следствие требования внешности агента. Ограничение прав доступа к странице памяти выполняется системным вызовом `mprotect`, однако этот вызов изменяет права того адресного пространства, из которого он исполнен. Агент же является отдельным процессом и обладает собственным

адресным пространством, поэтому прямой вызов `mprotect` из агента не затронул бы память цели. Чтобы снять право записи именно с области целевого процесса, агент вынужден добиться того, чтобы `mprotect` был исполнен самим целевым процессом. Это достигается приёмом, который далее именуется инъекцией системного вызова: агент временно подменяет состояние регистров цели так, чтобы она исполнила нужный вызов, после чего восстанавливает исходное состояние. Тем самым внешний характер агента сохраняется, а необходимое изменение прав производится в контексте цели.

Прежде чем установить охрану, агент разрешает объявленную область в её фактический виртуальный адрес. Поскольку область задаётся символически, разрешение выполняется в несколько шагов:

- 1) чтение карты отображений целевого процесса из интерфейса `/proc/<pid>/maps`;
- 2) определение базовых адресов загруженных модулей;
- 3) сопоставление имени символа с записью в таблице символов формата ELF соответствующего модуля;
- 4) вычисление фактического адреса как суммы базового адреса отображения и значения символа.

Такое вычисление обеспечивает корректную работу в условиях рандомизации адресного пространства, при которой абсолютный адрес заранее неизвестен. Размер охраняемой области при символическом задании определяется из той же таблицы символов; явное указание размера остаётся возможным, но не является обязательным. Полученный адрес выравнивается на границу страницы, так как ограничение прав средствами `mprotect` оперирует целыми страницами, а не отдельными байтами.

Собственно инъекция выполняется в виде последовательности обращений к `ptrace`:

- 1) чтение и сохранение текущего состояния регистров цели операцией `PTRACE_GETREGS`;
- 2) формирование изменённого состояния, в котором указатель инструкций направляется на заранее найденную в исполняемом отображении цели инструкцию системного вызова, а регистры заполняются номером вызова `mprotect` и его аргументами — адресом страницы, размером области и требуемой комбинацией прав;
- 3) запись изменённого состояния операцией `PTRACE_SETREGS`;

- 4) исполнение одной инструкции в пошаговом режиме, в результате которого системный вызов выполняется в собственном контексте цели;
- 5) чтение результата вызова из регистра-аккумулятора;
- 6) восстановление сохранённого ранее состояния регистров, возвращающее цель в исходное положение.

Использование уже присутствующей в коде цели инструкции системного вызова избавляет от необходимости записывать собственный код в чужое адресное пространство. Соответствующий фрагмент реализации приведён в листинге 3; полный текст модуля инъекции содержится в приложении (листинг A.2), а реализация установки и восстановления охраны, использующая инъекцию, — в листинге A.3.

```
// 1) сохранить регистры цели
pt_get_regs(pid, &saved);

// 2) подготовить регистры для вызова mprotect(addr, len, prot)
work = saved;
work.rip = syscall_insn;
work.rax = __NR_mprotect;           // номер системного вызова
work.rdi = addr;                   // 1-й аргумент: адрес
work.rsi = len;                    // 2-й аргумент: длина
work.rdx = (unsigned long)prot;     // 3-й аргумент: права
pt_set_regs(pid, &work);

// 3) исполнить одну инструкцию – это и будет syscall
pt_singlestep(pid, 0);
pt_wait(pid, &st);

// 4) прочитать результат mprotect из rax
pt_get_regs(pid, &work);
long ret = (long)work.rax;

// 5) восстановить исходные регистры цели
pt_set_regs(pid, &saved);
```

Рисунок 3 — Подмена регистров и исполнение инжектированного вызова mprotect (фрагмент inject_mprotect)

Приведённая последовательность применяется при установке охраны со снятием права записи, когда область переводится в режим, доступный только

для чтения. Та же последовательность с иной комбинацией прав используется и при временном восстановлении прав в цикле обработки нарушения, рассматриваемом далее. После исполнения инъекции страницы охраняемой области оказываются доступными лишь для чтения, вследствие чего любая последующая попытка записи в них вызывает аппаратное нарушение прав, передаваемое агенту в форме сигнала.

3.3.3 Цикл обработки нарушения и восстановление охраны

После установки охраны агент возобновляет исполнение цели и ожидает нарушения. Попытка записи в страницу, переведённую в режим только для чтения, порождает синхронное нарушение прав, которое ядро доставляет агенту как остановку цели с сигналом SIGSEGV. Получив такую остановку, агент считывает сопроводительную информацию о сигнале операцией `PTRACE_GETSIGINFO`, из которой извлекается адрес обращения, вызвавшего нарушение. По этому адресу принимается решение о дальнейшей обработке, различающее два случая:

- 1) обращение, попавшее в границы объявленного объекта, подлежит регистрации;
- 2) обращение к иным данным той же страницы, оказавшимся под охраной вследствие страничной гранулярности ограничения прав, регистрации не подлежит, но обрабатывается наравне с первым в части восстановления исполнения.

Для подлежащего регистрации обращения агент дополнительно считывает состояние регистров цели операцией `PTRACE_GETREGS`, получая адрес инструкции-инициатора и идентификатор потока, которые служат исходными данными для атрибуции.

Восстановление охраны после перехваченного обращения опирается на то же ограничение прав, что и её первичная установка, и обусловлено одноразовым характером срабатывания. Простое снятие права даёт лишь однократный сигнал: чтобы исполнение цели могло продолжиться, нарушившей инструкции необходимо дать завершиться, для чего право записи приходится временно вернуть. Восстановление выполняется последовательностью из трёх действий:

- 1) возврат права записи инъекцией `mprotect` с соответствующей комбинацией прав;

- 2) исполнение одной инструкции цели в пошаговом режиме операцией PTRACE_SINGLESTEP, дающее завершиться нарушившей записи;
- 3) повторное снятие права записи, восстанавливающее охрану.

После этого исполнение цели возобновляется, а агент возвращается к ожиданию следующего нарушения. Повторное снятие права на каждом цикле обеспечивает непрерывное детектирование обращений к области, а не однократную фиксацию первого из них.

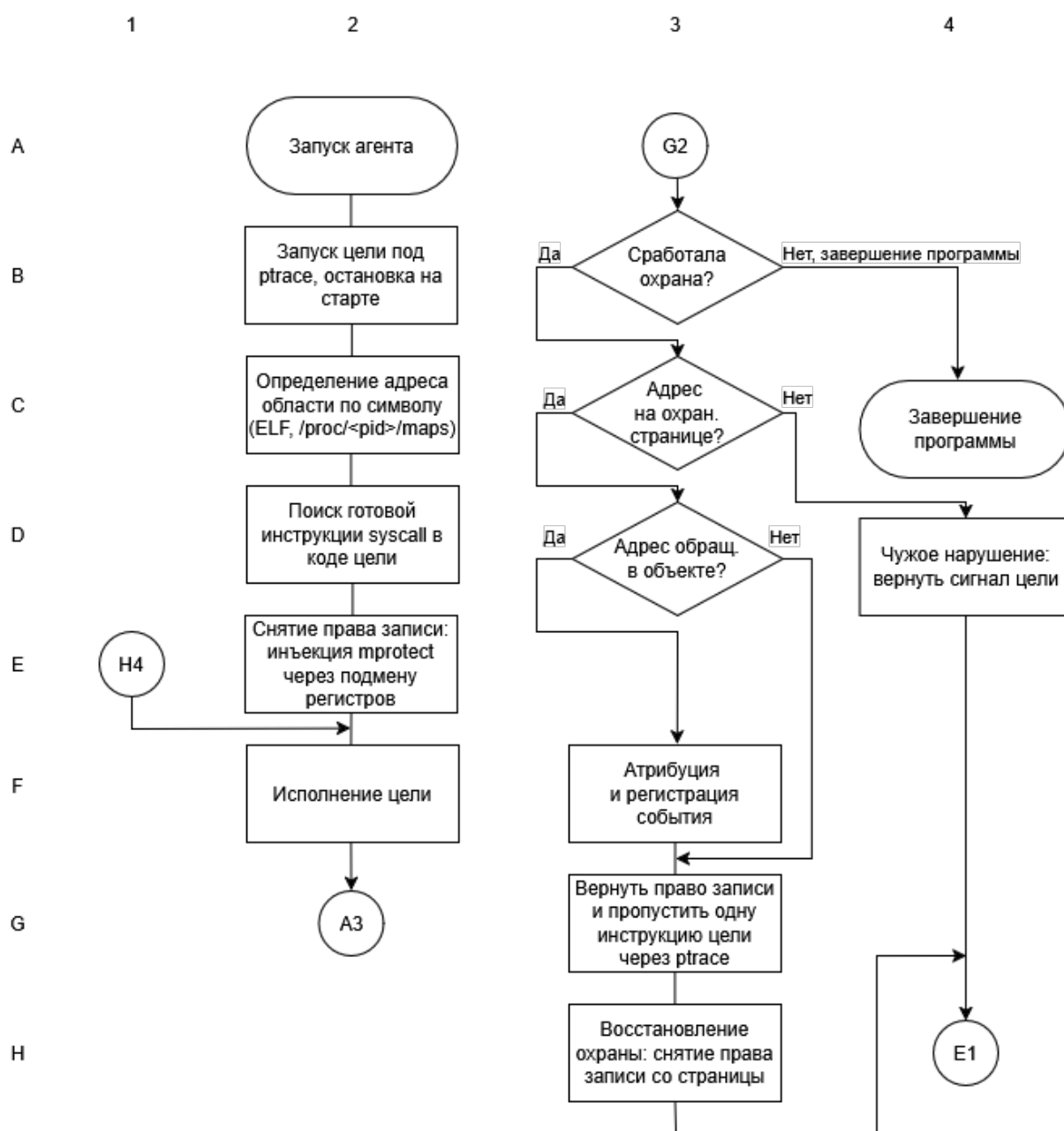


Рисунок 4 — Блок-схема цикла работы агента опытного образца

Приведённая схема раскрывает внутреннее устройство агента на уровне реализации, дополняя концептуальную блок-схему метода, представленную в предыдущей главе. В отличие от неё, схема отражает конкретные обращения к

примитивам ptrace и порядок их применения: однократную подготовительную часть, включающую запуск цели, разрешение области, поиск инструкции системного вызова и первичную установку охраны, и повторяющийся цикл, в котором перехват нарушения, чтение контекста, восстановление охраны и продолжение исполнения сменяют друг друга вплоть до завершения цели. Детализация шага атрибуции, изображённого на схеме единым блоком, и обсуждение различия легитимных и недопустимых обращений выносятся в последующие подразделы. Полный текст модуля агента, реализующего описанный цикл, приведён в приложении (листинг А.1).

3.4 Реализация атрибуции

Настоящий подраздел решает вторую из задач этапа — реализацию атрибуции — и описывает воплощение концептуального алгоритма, изложенного в предыдущей главе, в конкретные действия над картой памяти и таблицами символов целевого процесса. Если методика определяет атрибуцию как привязку перехваченного события к контексту выполнения и описывает её шаги безотносительно деталей реализации, то опытный образец сопоставляет каждому шагу обращение к определённому источнику данных операционной системы. Исходными данными атрибуции служат адрес инструкции-инициатора, считанный из состояния регистров цели в момент нарушения, и идентификатор потока; основная содержательная работа выполняется над адресом инструкции, поскольку именно он локализует обращение в коде программы. Результатом является привязка события к модулю и, при наличии необходимой символьной информации, к функции, в границы которой попадает адрес. Достижимая глубина привязки зависит от состава символьной информации модуля, что отражено в реализации явным образом и рассматривается в конце подраздела.

Первым шагом агент считывает карту отображений целевого процесса из интерфейса `/proc/<pid>/maps`. Каждая строка карты разбирается в запись, содержащую границы области виртуального адресного пространства, права доступа и связанный с областью путь к файлу — исполняемому образу, разделяемой библиотеке или псевдониму вида `[heap]` и `[stack]`. Совокупность таких записей образует отображение адресного пространства цели на момент нарушения и служит основой для последующего определения модуля. Поскольку одному модулю в карте обычно соответствует несколько смежных

областей с различными правами, агент дополнительно определяет базовый адрес загрузки модуля как наименьший из начальных адресов областей, относящихся к одному и тому же файлу. Базовый адрес необходим для перехода от абсолютного виртуального адреса к смещению внутри модуля.

Определив модуль, в чьи границы попадает адрес инструкции-инициатора, агент вычисляет смещение этого адреса относительно базового адреса модуля. Смещение, в отличие от абсолютного адреса, не зависит от рандомизации адресного пространства: при каждом запуске модуль загружается по иному базовому адресу, тогда как расположение инструкции внутри модуля остаётся неизменным. Это свойство делает смещение устойчивой основой для дальнейшего сопоставления с таблицей символов, которая хранит адреса символов в терминах, фиксированных на этапе компоновки. Вычисленные на этом шаге модуль и смещение уже сами по себе достаточны для локализации источника обращения с точностью до компонента программы и образуют нижнюю границу результата атрибуции.

Разрешение смещения в имя функции выполняется чтением таблицы символов формата ELF из файла модуля. Агент просматривает секции полной таблицы символов и таблицы динамических символов и среди символов типа функции отыскивает тот, в границы которого — от его значения до значения, увеличенного на размер символа, — попадает вычисленное смещение. При этом учитывается тип объектного файла:

- 1) для исполняемого файла типа ET_EXEC значение символа задаёт абсолютный виртуальный адрес;
- 2) для разделяемого объекта типа ET_DYN, к которому относятся и позиционно-независимые исполняемые файлы, значение символа выражено как смещение от базы загрузки.

Приведение обоих случаев к единому смещению внутри модуля обеспечивает корректное сопоставление независимо от способа сборки целевой программы. При успешном сопоставлении к результату атрибуции добавляется имя найденной функции.

Если символьная информация в модуле отсутствует — например, у модуля, собранного без таблицы символов, — разрешение функции невозможно, и атрибуция ограничивается ранее определёнными модулем и смещением. Этот случай обрабатывается в реализации не как ошибочный, а как штатный исход с пониженной глубиной привязки: запись о событии формируется в любом случае, а отсутствие имени функции отмечается явно.

Тем самым граница достижимой точности, обоснованная в предыдущей главе как принципиальное свойство метода, а не дефект реализации, воспроизводится и в опытном образце: при наличии символьной или отладочной информации атрибуция доводится до имени функции, при её отсутствии — остаётся определённой на уровне модуля и смещения, что само по себе локализует источник обращения с точностью до компонента программы.

Результат атрибуции оформляется в виде записи о событии — структуры фиксированного состава, материализующей требования к составу записи, сформулированные на предыдущем этапе. Каждая запись содержит порядковый номер события, идентификатор потока-инициатора, тип контролируемого доступа, адрес обращения к охраняемой области, адрес инструкции-инициатора, а также результат атрибуции — модуль со смещением и, при наличии символьной информации, имя функции. Образец записи, формируемой агентом при перехвате недопустимого обращения, приведён в листинге 5.

```
событие #1 | TID=134139 | доступ=W
адрес обращения : 0x63abaa15b018
инструкция (RIP): 0x713da1782109
модуль          : <путь>/libs2attacker.so + 0x1109
функция         : attacker_write
```

Рисунок 5 — Формат записи о перехваченном событии доступа

В приведённой записи поле адреса обращения указывает на байт охраняемой области, к которому произошло обращение, тогда как адрес инструкции-инициатора (RIP) локализует совершивший обращение участок кода; именно последний служит входом алгоритма атрибуции, результат которого представлен полями модуля со смещением и имени функции. Смещение внутри модуля приводится наряду с именем функции, поскольку остаётся определённым и в отсутствие символьной информации, тогда как поле функции в этом случае опускается. Тем самым запись о событии отражает достигнутую глубину атрибуции явно: от модуля и смещения как нижней границы до имени функции при наличии необходимых символьных данных. Полный текст модуля атрибуции приведён в приложении (листинг А.4).

3.5 Выявленные технические проблемы и корректировка методики

Настоящий подраздел решает третью из задач этапа — документирование технических проблем, выявленных при разработке опытного образца, и корректировку методики по результатам этой разработки. Поскольку опытный образец создавался именно для проверки гипотез о работоспособности выбранного механизма, обнаруженные при его реализации затруднения представляют собой содержательный результат, а не свидетельство недоработки: они показывают, в какой мере методика, сформулированная на уровне метода, выдерживает столкновение с конкретной реализацией. Проблемы рассматриваются в единой схеме: для каждой указывается, была ли она предусмотрена методикой и подтвердила её, потребовала ли уточнения метода, или относится к направлениям дальнейшей работы, не затрагивая корректности базового цикла. Часть из перечисленных проблем проявляется и в результатах испытаний; соответствующие наблюдения на модельных сценариях приводятся в главе, посвящённой валидации, тогда как здесь рассматривается их связь с методикой и реализацией.

При разработке опытного образца выявлены пять технических проблем, по-разному соотносящихся с методикой:

- 1) хрупкость инъекции системного вызова — подтверждает методику и заложенные в неё риски, не требуя изменения метода;
- 2) неразличимость легитимной записи на этапе инициализации — потребовала уточнения методики введением различия обращений по контексту инициатора;
- 3) одношаговое восстановление охраны и сопутствующая гонка — в части одноразовости срабатывания подтверждает методику, в части гонки отнесена к направлениям дальнейшей работы;
- 4) потеря охраны при изменении карты памяти — отнесена к направлениям дальнейшей работы, не затрагивая базового цикла мониторинга;
- 5) ограничения среды исполнения — уточняют условия применимости методики в разных режимах работы.

Далее каждая из перечисленных проблем рассматривается подробнее.

Хрупкость инъекции системного вызова выявлена непосредственно при реализации установки охраны. Подмена состояния регистров целевого

процесса с последующим исполнением одной инструкции в чужом контексте является технически хрупкой операцией: она опирается на сохранение и точное восстановление полного состояния регистров, на присутствие в исполняемом отображении цели готовой инструкции системного вызова и на корректную интерпретацию результата вызова. Любое отклонение — неполное восстановление состояния, ошибка в наборе аргументов, отсутствие подходящей инструкции — нарушает дальнейшее исполнение цели. Эта проблема не противоречит методике и не требует её изменения, поскольку методика сознательно отвлекается от деталей программной реализации примитива ограничения прав. Вместе с тем она подтверждает обоснованность отнесения повышенных накладных расходов и сложности реализации к рискам выбранного механизма: инъекция оказывается платой за внешний характер агента, не имеющего иного способа изменить права чужого адресного пространства. Проблема разрешена в пределах опытного образца и определяет требования к аккуратности реализации, но не выходит за рамки принятого метода.

Неразличимость легитимной записи на этапе инициализации носит концептуальный характер и потребовала уточнения методики. В режиме запуска цели под контролем агента охрана устанавливается в самом начале исполнения, до того как целевая программа выполнит инициализацию защищаемой области. Вследствие этого первая, легитимная запись в область — её инициализация — перехватывается наравне с недопустимыми обращениями, поскольку по самому факту записи отличить инициализацию от нарушения невозможно. Методика в исходном виде предполагала, что момент фиксации области как неизменяемой известен, однако не определяла, каким образом этот момент задаётся в режиме запуска. Для устранения этого пробела метод дополнен различием обращений по контексту инициатора: запись, исходящая из заранее объявленного набора функций, которым изменение области разрешено, признаётся легитимной и не регистрируется как нарушение, тогда как запись из прочих функций трактуется как недопустимая. Такое различие опирается на уже реализованную атрибуцию инициатора и согласуется с заложенным в методике направлением развития, при котором часть записей признаётся легитимной в зависимости от контекста. Введённое различие встраивается в ранее описанный цикл на шаге формирования записи о событии и не изменяет остальной его структуры, отражённой на блок-схеме: проверка контекста инициатора лишь

определяет, формируется ли запись для обращения, попавшего в границы охраняемого объекта. Тем самым проблема приводит к уточнению методики, не меняя её базового цикла; проявление этой проблемы и действие введённого различия демонстрируются на соответствующем модельном сценарии в главе, посвящённой валидации.

Одношаговое восстановление охраны и сопутствующая гонка охватывают два связанных следствия выбранного способа восстановления охраны. Первое — одноразовость срабатывания механизма ограничения прав: после перехвата нарушения для продолжения исполнения право доступа приходится временно вернуть, вследствие чего охрана на короткий интервал снимается, и для её сохранения требуется повторное снятие права после одиночного шага. Это следствие предусмотрено методикой, заложено в её цикл и в реализации подтверждено: непрерывное детектирование обеспечивается именно повторным снятием права на каждом проходе цикла. Второе следствие — возможность гонки в интервале между возвратом и повторным снятием права: в многопоточной цели обращение к охраняемой области из другого потока в этот момент могло бы пройти в обход контроля. В опытном образце это следствие снято принятым допущением об однопоточности цели и потому не проявляется, однако в общем случае оно представляет собой отдельную задачу синхронизации. Таким образом, в части одноразовости проблема подтверждает методику, а в части гонки относится к направлениям дальнейшей работы, обозначенным при анализе рисков выбранного механизма, и не затрагивает корректности цикла в пределах принятого допущения.

Потеря охраны при изменении карты памяти относится к направлениям дальнейшей работы и в опытном образце не разрешена. Ограничение прав устанавливается на конкретные страницы виртуального адресного пространства цели, определённые на момент установки охраны. Если целевой процесс изменяет карту своей виртуальной памяти — переотображает или освобождает охраняемую область, — установленные ограничения прав могут быть утрачены, а охрана тем самым снята без ведома агента. Для устойчивой работы в общем случае агент должен отслеживать изменения карты отображений цели и переустанавливать охрану при необходимости. Эта проблема была обозначена при анализе рисков выбранного механизма в предыдущей главе и подтверждается на уровне реализации; в опытном образце она не закрывается, поскольку используемые модельные сценарии не изменяют карту памяти в течение наблюдения. Соответствующая обработка

переотображения относится к дальнейшему развитию инструмента и не затрагивает корректности базового цикла мониторинга в рамках текущего этапа.

Ограничения среды исполнения носят эксплуатационный характер и определяются внешними по отношению к инструменту условиями. Установление контроля над целевым процессом средствами ptrace ограничено политиками безопасности операционной системы: доступ к трассировке регулируется параметром ядра, а трассировка стороннего процесса в общем случае требует повышенных привилегий. В принятом для опытного образца режиме запуска цели под контролем агента, при котором агент сам порождает наблюдаемый процесс, этих ограничений обычно достаточно избежать, поскольку трассировка собственного потомка допускается без дополнительных привилегий. Однако режим подключения к уже выполняющемуся процессу, отнесённый к направлениям дальнейшей работы, потребует учёта указанных политик и, как правило, повышенных привилегий. Эта проблема согласуется со сформулированным на предыдущем этапе эксплуатационным требованием, допускающим необходимость повышенных привилегий, и потому не противоречит методике, а уточняет условия её применимости в разных режимах работы.

3.6 Выводы

В настоящей главе решена вторая задача этапа — разработан опытный образец модуля перехвата событий доступа к памяти. Изложение велось от назначения и границ образца к реализации составляющих его механизмов и далее к выявленным при разработке техническим проблемам, в результате чего методика контекстного мониторинга, сформулированная в предыдущей главе на уровне метода, получила воплощение в работающем прототипе и проверку реальной реализацией. Образец охватывает три составляющие, отвечающие задачам этапа: реализацию перехвата нарушений доступа, реализацию атрибуции перехваченного события и документирование технических проблем с уточнением методики.

Основные результаты главы состоят в следующем:

- 1) реализован механизм перехвата. Шаги методики сопоставлены конкретным примитивам операционной системы: установление контроля над целью — порождению трассируемого процесса

средствами `ptrace` в режиме запуска под контролем агента; ограничение прав — инъекции системного вызова `mprotect` в адресное пространство цели, разрешающей противоречие между требованием внешности агента и тем, что `mprotect` исполняется самим целевым процессом; отлов нарушения и восстановление охраны — перехвату сигнала с последующим одношаговым исполнением и повторным снятием права записи, обеспечивающим непрерывное детектирование;

- 2) реализована атрибуция. Концептуальный алгоритм привязки события к контексту выполнения воплощён в последовательность действий над картой отображений и таблицами символов целевого процесса: определение модуля и устойчивого к рандомизации смещения по карте памяти и разрешение смещения в имя функции по символьной информации формата ELF, с корректной обработкой различных типов объектного файла и штатной деградацией до модуля и смещения при отсутствии символов;
- 3) документированы выявленные технические проблемы и уточнена методика. Пять обнаруженных при разработке проблем соотнесены с методикой: хрупкость инъекции и одноразовость срабатывания подтверждают методику и заложенные в неё риски; неразличимость легитимной записи на этапе инициализации потребовала уточнения метода введением различия обращений по контексту инициатора; гонка при одношаговом восстановлении, потеря охраны при переотображении области и учёт политик трассировки отнесены к направлениям дальнейшей работы, не затрагивая корректности базового цикла.

Тем самым подтверждены предположения, на проверку которых был направлен опытный образец: внешний перехват записи в охраняемую область реализуем на немодифицированном ядре, атрибуция перехваченного события достижима вплоть до иницировавшей его инструкции, а заложенный в методику цикл — ограничение прав, отлов нарушения, атрибуция и восстановление охраны — работоспособен в составе единого прототипа. Разработка образца не только подтвердила работоспособность выбранного механизма, но и уточнила методику в том пункте, где её исходная формулировка оставляла пробел. Полученный прототип образует основу для решения третьей задачи этапа — количественной проверки образца на

модельных сценариях доступа к памяти, составляющей предмет следующей главы.

4 Валидация опытного образца на модельных сценариях доступа к памяти

Настоящая глава решает третью задачу этапа — проверку разработанного опытного образца на модельных сценариях доступа к памяти. Если предыдущая глава описывала устройство образца, то есть сопоставление шагов методики конкретным примитивам операционной системы, то настоящая глава обращается к его поведению: она устанавливает, приводит ли реализованный цикл мониторинга к ожидаемым исходам на контролируемом наборе обращений к охраняемой области. Проверка опирается на образец в том виде, в каком он представлен в предыдущей главе, и не вводит новых механизмов. В основу проверки положены три свойства методики, выделенные при определении подхода к валидации: корректность отлова недопустимых обращений, отсутствие ложных срабатываний на легитимном пути исполнения и базовая атрибуция инициатора обращения. Каждое из этих свойств проверяется одним или несколькими модельными сценариями, для каждого из которых заранее определён ожидаемый исход. Изложение организовано следующим образом: сначала описываются методика и условия проведения испытаний, затем приводятся модельные сценарии и результаты их прогона, после чего даётся первичная оценка накладных расходов; обобщение результатов и границы их достоверности рассматриваются в завершающих подразделах.

4.1 Методика и условия проведения испытаний

Все сценарии исполняются в единых условиях, что обеспечивает сопоставимость их результатов. Испытания проводятся на архитектуре x86-64 под управлением Linux с немодифицированным ядром, с использованием стандартной библиотеки glibc; целевые программы собираются штатным компилятором без флагов инструментализации, в соответствии с принятыми в предыдущей главе характеристиками среды. Образец запускается в режиме порождения цели под своим контролем — единственном режиме, реализованном на текущем этапе, — а целевые программы являются однопоточными, что соответствует принятому модельному допущению об однопоточности.

Запуск образца выполняется командой, в которой указываются способ задания охраняемой области и путь к целевой программе. В типичном случае

область задаётся символически, по имени соответствующей переменной; для сценария, в котором целевая программа собрана как позиционно-зависимая, область задаётся абсолютным адресом, печатаемым самой программой. Наблюдаемым выходом образца служит журнал событий, формируемый агентом при перехвате обращений к охраняемой области. Каждая запись журнала имеет формат, описанный в предыдущей главе (листинг 5), и содержит идентификатор потока-инициатора, тип доступа, адрес обращения, адрес инструкции-инициатора и результат атрибуции — модуль со смещением и, при наличии символьной информации, имя функции. Отсутствие записей по завершении прогона является столь же значимым наблюдением, как и их наличие, поскольку часть сценариев проверяет именно отсутствие срабатываний.

Исход каждого сценария оценивается сопоставлением фактического выхода образца с ожидаемым, заданным до прогона. Ожидаемый исход формулируется в терминах двух характеристик: факта регистрации события (зарегистрировано или нет) и требуемой глубины атрибуции (до модуля или до функции) для тех сценариев, в которых событие ожидается. Сценарий считается пройденным, если фактический выход совпал с ожидаемым по обоим характеристикам. Соответствие модельных сценариев проверяемым свойствам методики и ожидаемым исходам приведено в таблице 2.

Таблица 2 — Соответствие модельных сценариев проверяемым свойствам методики

Сценарий	Проверяемое свойство	Ожидаемый исход
Легитимная запись	Отсутствие ложных срабатываний	Событие не регистрируется
Запись из постороннего модуля	Корректность отлова, атрибуция	Событие; атрибуция до модуля
Запись из посторонней функции	Корректность отлова, атрибуция	Событие; атрибуция до функции
Переполнение соседнего буфера	Корректность отлова	Событие; атрибуция до функции
Обращение к незащищённой области	Избирательность охраны	Событие не регистрируется

Приведённый в таблице набор сценариев покрывает три выделенных свойства методики: корректность отлова проверяется сценариями с недопустимыми обращениями (из постороннего модуля, из посторонней

функции и вследствие переполнения), отсутствие ложных срабатываний — сценарием легитимной записи и контрольным сценарием обращения к незащищённой области, а базовая атрибуция — сценариями, в которых требуется указать инициатора с точностью до модуля либо до функции. Контрольный сценарий обращения к незащищённой области выделен особо: он проверяет не отлов нарушения, а избирательность охраны, то есть отсутствие реакции на обращения к областям, не помещённым под защиту. Описанные в настоящем подразделе единые условия и критерий зачёта применяются ко всем сценариям, описание и результаты прогона которых рассматриваются далее.

4.2 Модельные сценарии

Настоящий подраздел описывает пять модельных целевых программ, на которых проверяется образец. Для каждой приводится постановка — какое обращение к охраняемой области в ней происходит и какое свойство методики она проверяет — и сокращённый фрагмент кода, порождающий это обращение. Фрагменты приведены в сокращении: из каждой целевой программы оставлены лишь объявление охраняемой переменной и обращение к ней, тогда как вспомогательный код (включение заголовков, диагностическая печать) опущен. Все программы являются однопоточными и порождают обращение к охраняемой области детерминированно, что обеспечивает воспроизводимость прогонов. Результаты прогона приведённых сценариев рассматриваются в следующем подразделе.

Первый сценарий — легитимная запись — проверяет отсутствие ложных срабатываний на законном пути исполнения. Целевая программа содержит критическую переменную и единственное обращение к ней: инициализирующую запись из функции `main`, которая по условию сценария является допустимой. Особенность сценария в том, что в режиме запуска цели под контролем агента охрана устанавливается до начала исполнения целевого кода, то есть до инициализирующей записи, вследствие чего по самому факту записи инициализация неотличима от нарушения. Сценарий предназначен для проверки введённого в предыдущей главе различения обращений по контексту инициатора.

```
volatile uint64_t critical_flag = 0;

int main(void) {
    critical_flag = 1;    // легитимная инициализация
    return 0;
}
```

Рисунок 6 — Сценарий легитимной записи (сокращённо)

Второй сценарий — запись из постороннего модуля — проверяет корректность отлова и атрибуцию на уровне модуля. Критическая переменная объявлена в основной программе, а запись в неё производится функцией `attacker_write`, размещённой в отдельной разделяемой библиотеке, которой изменение переменной не положено. Сценарий построен так, чтобы охраняемый объект и инициатор обращения принадлежали разным модулям, что позволяет проверить, отделяет ли атрибуция место расположения объекта от места, откуда исходит обращение.

```
// в библиотеке-нарушителе:
void attacker_write(volatile uint64_t *target) {
    *target = 0xBADC0DE;    // запись из чужого модуля
}

// в основной программе:
volatile uint64_t critical_flag = 0;
attacker_write(&critical_flag);
```

Рисунок 7 — Сценарий записи из постороннего модуля (сокращённо)

Третий сценарий — запись из посторонней функции того же модуля — проверяет способность атрибуции различать инициатора на уровне функции, а не только модуля. Критическая переменная и нарушающая функция `writer_foreign` расположены в одном модуле; запись в переменную производится не из легитимного пути, а из этой посторонней функции. В отличие от предыдущего сценария, здесь инициатор и охраняемый объект принадлежат одному модулю, поэтому различение возможно лишь на уровне функции.

```
volatile uint64_t critical_flag = 0;

void writer_foreign(uint64_t v) {
    critical_flag = v;    // нарушение из посторонней функции
}
```

Рисунок 8 — Сценарий записи из посторонней функции (сокращённо)

Четвёртый сценарий — запись вследствие переполнения соседнего буфера — воспроизводит типовой механизм эксплуатации, при котором запись за границу буфера затирает смежные управляющие данные. Охраняемое поле и переполняемый буфер размещены как соседние поля одной структуры, что делает затирание поля при выходе записи за границу буфера детерминированным. Поскольку для этого сценария требуется стабильный между запусками адрес поля, целевая программа собрана как позиционно-зависимая (без рандомизации размещения), а охраняемая область задаётся агенту абсолютным адресом; причины такого решения и его связь с символическим заданием области рассмотрены в предыдущей главе.

```
static struct {
    char    overflow_buf[8];
    uint64_t flag;           // охраняемое поле, идёт сразу за буфером
} blk;

// запись за границу overflow_buf затирает blk.flag
memset(blk.overflow_buf, 0x41, sizeof(blk.overflow_buf) + 8);
```

Рисунок 9 — Сценарий переполнения соседнего буфера (сокращённо)

Пятый сценарий — контрольное обращение к незащищённой области — проверяет избирательность охраны. Целевая программа содержит две переменные: критическую, помещаемую под охрану, и постороннюю, охраной не покрываемую. Программа производит запись только в постороннюю переменную, не обращаясь к охраняемой. Сценарий проверяет, что обращение к не помещённой под охрану памяти не порождает событий, то есть что охрана действует избирательно и не реагирует на обращения за пределами объявленной области.


```

volatile uint64_t critical_flag = 0;    // под охраной
volatile uint64_t other_var      = 0;    // не под охраной

int main(void) {
    other_var = 0xAABBCCDD;    // запись мимо охраны
    return 0;
}

```

Рисунок 10 — Контрольный сценарий обращения к незащищённой области (сокращённо)

Перечисленные пять сценариев в совокупности покрывают три проверяемых свойства методики и заданы так, чтобы порождать обращения к памяти воспроизводимо и детерминированно. Этим обеспечивается однозначность сопоставления фактического исхода прогона с ожидаемым, к которому переходит следующий подраздел.

4.3 Результаты прогона модельных сценариев

Настоящий подраздел приводит результаты прогона образца по пяти описанным сценариям. Для каждого сценария приводится полученный от агента вывод и его сопоставление с ожидаемым исходом, заданным ранее. Выводы приведены в том виде, в каком их формирует агент; обобщены лишь пути к исполняемым файлам и модулям. Конкретные значения адресов специфичны для отдельного прогона и от запуска к запуску меняются вследствие рандомизации адресного пространства, тогда как смещение внутри модуля остаётся устойчивым; именно поэтому существенны не абсолютные адреса, а факт регистрации события и достигнутая глубина атрибуции. Порядок рассмотрения сценариев соответствует их порядку в предыдущем подразделе.

4.3.1 Результаты запусков сценариев

Сценарий легитимной записи прогонялся дважды. При запуске без указания функций, которым запись разрешена, инициализирующая запись из `main` регистрируется как нарушение: агент перехватывает обращение и формирует запись о событии, относя инициатора к функции `main`. Этот исход воспроизводит проблему неразличимости легитимной записи на этапе инициализации, описанную в предыдущей главе.

```
$ memwatch --symbol critical_flag -- ./bin/sl_legit_write
agent: охрана области 0x5c5c6af58018 (8 байт), страницы
0x5c5c6af58000..+0x1000
событие #1 | TID=134010 | доступ=W
адрес обращения : 0x5c5c6af58018
инструкция (RIP): 0x5c5c6af55131
модуль           : ./bin/sl_legit_write + 0x1131
функция          : main
agent: цель завершилась с кодом 0
```

Рисунок 11 — Сценарий легитимной записи: срабатывание без списка разрешённых функций

При повторном запуске с указанием функции `main` как разрешённой обращение, исходящее из неё, признаётся легитимным и не регистрируется; по завершении прогона журнал событий пуст. Тем самым различие по контексту инициатора устраняет ложное срабатывание на инициализации, не затрагивая способности образца перехватывать недопустимые обращения из прочих функций. Полученный исход соответствует ожидаемому: при отнесении инициализирующей функции к разрешённым ложное срабатывание отсутствует.

```
$ memwatch --symbol critical_flag --allow main -- ./bin/
sl_legit_write
agent: охрана области 0x599f9f0ac018 (8 байт), страницы
0x599f9f0ac000..+0x1000
agent: цель завершилась с кодом 0
```

Рисунок 12 — Сценарий легитимной записи: обращение признано легитимным по контексту инициатора

Различие по контексту инициатора опирается на атрибуцию по адресу инструкции и представляет собой эвристику уровня функции: оно отсеивает обращения по принадлежности инициатора к разрешённому набору функций, но не подтверждает легитимность обращения иными средствами. Границы применимости такого различия относятся к вопросам, рассматриваемым при обсуждении достоверности результатов.

Сценарий записи из постороннего модуля показывает, что обращение перехвачено, а атрибуция отнесла инициатора к разделяемой библиотеке и функции `attacker_write` в ней, тогда как сама переменная расположена в

основной программе. Полученный исход соответствует ожидаемому: событие зарегистрировано, инициатор определён с точностью до модуля и функции. Существенно, что атрибуция отделяет место расположения охраняемого объекта от места, откуда исходит обращение: переменная принадлежит основной программе, но нарушителем верно назван посторонний модуль.

```
$ memwatch --symbol critical_flag -- ./bin/s2_foreign_module
agent: охрана области 0x63abaa15b018 (8 байт), страницы
0x63abaa15b000..+0x1000
[s2] critical_flag = 0x0, зовём чужой модуль
событие #1 | TID=134139 | доступ=W
адрес обращения : 0x63abaa15b018
инструкция (RIP): 0x713da1782109
модуль           : ./bin/libs2attacker.so + 0x1109
функция          : attacker_write
[s2] после чужого модуля = 0xbadc0de
agent: цель завершилась с кодом 0
```

Рисунок 13 — Сценарий записи из постороннего модуля: атрибуция указывает на библиотеку-нарушитель

Сценарий записи из посторонней функции того же модуля показывает, что обращение перехвачено, а атрибуция указала на функцию-нарушитель `writer_foreign`. Поскольку охраняемый объект и инициатор в этом сценарии принадлежат одному модулю, верное определение функции подтверждает, что атрибуция различает инициатора на уровне функции, а не ограничивается модулем. Полученный исход соответствует ожидаемому.

```
$ memwatch --symbol critical_flag -- ./bin/s3_foreign_func
agent: охрана области 0x5e8cb06fe018 (8 байт), страницы
0x5e8cb06fe000..+0x1000
[s3] critical_flag = 0x0
событие #1 | TID=134206 | доступ=W
адрес обращения : 0x5e8cb06fe018
инструкция (RIP): 0x5e8cb06fb159
модуль           : ./bin/s3_foreign_func + 0x1159
функция          : writer_foreign
[s3] после writer_foreign = 0xf00d
```

Рисунок 14 — Сценарий записи из посторонней функции: атрибуция различает инициатора на уровне функции

Сценарий переполнения соседнего буфера показывает, что затирающая запись, вышедшая за границу буфера, перехвачена как обращение к охраняемому полю, а инициатор отнесён к функции `main`, выполнившей переполнение. Тем самым типовой механизм эксплуатации, при котором управляющие данные затираются переполнением смежного буфера, обнаруживается и атрибутируется наравне с прямой записью в охраняемую область. В отличие от прочих сценариев, охраняемая область задана здесь абсолютным адресом, поскольку целевая программа собрана как позиционно-зависимая; этот адрес печатается самой программой и передаётся агенту. Полученный исход соответствует ожидаемому.

```
$ memwatch --addr 0x404038 --len 8 -- ./bin/s4_buffer_overflow
agent: охрана области 0x404038 (8 байт), страницы 0x404000..+0x1000
событие #1 | TID=135685 | доступ=W
  адрес обращения : 0x404038
  инструкция (RIP): 0x40115e
  модуль           : ./bin/s4_buffer_overflow + 0x115e
  функция          : main
[s4] адрес flag = 0x404038
[s4] flag до переполнения = 0x0
[s4] flag после переполнения = 0x4141414141414141
agent: цель завершилась с кодом 0
```

Рисунок 15 — Сценарий переполнения соседнего буфера: затирание поля перехвачено

Контрольный сценарий обращения к незащищённой области показывает, что запись в постороннюю, не помещённую под охрану переменную не порождает ни одного события: по завершении прогона журнал событий пуст. Этим подтверждается избирательность охраны — образец реагирует лишь на обращения к объявленной области и не вмешивается в обращения к прочей памяти процесса. Полученный исход соответствует ожидаемому.

```
$ memwatch --symbol critical_flag -- ./bin/s5_control_unguarded
agent: охрана области 0x5a8cf4938018 (8 байт), страницы
0x5a8cf4938000..+0x1000
[s5] other_var = 0xaabbccdd, critical_flag не тронут
agent: цель завершилась с кодом 0
```

Рисунок 16 — Контрольный сценарий: обращение к незащищённой области не порождает событий

Прогон всех пяти сценариев дал исходы, совпавшие с ожидаемыми: недопустимые обращения перехвачены и атрибутированы с требуемой

глубиной, легитимная инициализация при включённом различении по контексту не вызвала ложного срабатывания, а обращение к незащищённой области осталось без реакции. Помимо функциональных свойств, на отдельной модельной цели измерены накладные расходы, вносимые контролем доступа, чему посвящён следующий подраздел.

4.3.2 Оценка накладных расходов

Помимо проверки функциональных свойств, опытный образец позволяет получить первичную оценку накладных расходов, вносимых контролем доступа. Прямое измерение затруднено тем, что основные модельные сценарии порождают по одному обращению к охраняемой области каждый, тогда как для устойчивой оценки стоимости перехвата требуется многократное срабатывание. Поэтому использовалась отдельная модельная цель, совершающая заданное число записей в охраняемую переменную в плотном цикле; каждая запись порождает полный цикл обработки нарушения, а суммарное время, делённое на число итераций, даёт среднюю стоимость одного срабатывания. Время измерялось самой целью по монотонному таймеру, что исключает из оценки стоимость запуска агента, и сверялось с внешним измерением полного времени исполнения. Цель собрана без оптимизаций, что исключает устранение цикла компилятором.

Измерение проводилось в трёх режимах: исполнение цели без агента, исполнение под агентом с регистрацией каждого нарушения и исполнение под агентом с отнесением записи к легитимной по контексту инициатора, без формирования записи о событии. Каждый режим под агентом измерялся по тридцати прогонам; приводятся медиана и выборочное стандартное отклонение. Усреднённые результаты приведены в таблице 3.

Таблица 3 — Стоимость одного обращения к охраняемой области в разных режимах

Режим исполнения	Время на одно обращение
Без агента	2 нс
Под агентом, нарушение регистрируется	223 ± 12 мкс
Под агентом, обращение признано легитимным	217 ± 21 мкс

Из приведённых данных следует несколько наблюдений. Стоимость одного обращения к охраняемой области под наблюдением составляет около 220 мкс, тогда как то же обращение без наблюдения занимает единицы наносекунд; основной вклад в эту величину вносят переключения

контекста между агентом и целью, операции ptrace по чтению состояния и инъекция вызова mprotect с одиночным шагом. Стоимость самой атрибуции и формирования записи на этом фоне статистически неразличима: медианы режимов с регистрацией нарушения и с отнесением обращения к легитимному различаются менее чем на величину выборочного стандартного отклонения, то есть введённое различие по контексту инициатора не увеличивает накладных расходов и влияет лишь на объём формируемого журнала. Внешнее измерение полного времени исполнения отличается от суммарного времени цикла на единицы миллисекунд, относимые к однократной подготовительной фазе — запуску цели, разрешению символа и первичной установке охраны, — что подтверждает корректность самозамера.

Полученную оценку следует трактовать с учётом природы использованной цели, обращающейся к охраняемой области в плотном цикле. Такой режим представляет собой наихудший для предложенного подхода случай и противоположен условиям, для которых инструмент предназначен: накладные расходы линейны по числу срабатываний, а стоимость одного срабатывания от частоты обращений не зависит, тогда как наблюдение, не сопровождающееся нарушениями, практически бесплатно. Избирательность охраны, обоснованная при анализе рисков выбранного механизма, снижает накладные расходы не удешевлением отдельного срабатывания, а сведением их числа к малому: на реальной цели, обращающейся к защищаемому секрету или флагу единицы раз за время исполнения, суммарные накладные расходы составляют доли миллисекунды и практически неощутимы. Тем самым измерение подтверждает предположение о приемлемости накладных расходов в условиях избирательной охраны, сформулированное при выборе механизма перехвата.

4.4 Обобщение результатов и границы валидации

Рассмотренные в предыдущем подразделе прогоны позволяют вынести суждение о методике в целом, прежде чем очертить пределы, в которых это суждение справедливо. Все пять модельных сценариев дали исходы, совпавшие с заданными до прогона ожиданиями, что сводит результаты испытаний к единой картине: реализованный цикл мониторинга перехватывает недопустимые обращения к охраняемой области, не порождает ложных срабатываний на легитимном пути и атрибутирует инициатора с требуемой

глубиной. Совпадение наблюдалось по обеим характеристикам ожидаемого исхода — как по факту регистрации события, так и по достигнутой глубине атрибуции. Сводка ожидаемых и фактических исходов по всем сценариям приведена в таблице 4.

Таблица 4 — Сводка результатов прогона модельных сценариев

Сценарий	Фактический исход	Соответствие
Легитимная запись (с различием)	Событие не зарегистрировано	соответствует
Запись из постороннего модуля	Событие; атрибуция до модуля и функции	соответствует
Запись из посторонней функции	Событие; атрибуция до функции	соответствует
Переполнение соседнего буфера	Событие; атрибуция до функции	соответствует
Обращение к незащищённой области	Событие не зарегистрировано	соответствует

В терминах трёх свойств методики результаты распределяются следующим образом. Корректность отлова недопустимых обращений подтверждена тремя сценариями с нарушениями — записью из постороннего модуля, записью из посторонней функции и переполнением соседнего буфера, — в каждом из которых обращение было перехвачено. Отсутствие ложных срабатываний подтверждено сценарием легитимной записи при включённом различии по контексту и контрольным сценарием обращения к незащищённой области, в обоих из которых журнал событий остался пуст. Базовая атрибуция подтверждена на двух уровнях глубины: до модуля — сценарием записи из посторонней библиотеки, до функции — сценариями записи из посторонней функции и переполнения. В пределах принятых допущений ожидаемая полнота обнаружения является полной: всякое недопустимое обращение на запись к охраняемой области, совершённое в рамках однопоточной модели, подлежит перехвату, что согласуется с оценкой полноты, данной при определении подхода к валидации.

Полученные результаты справедливы в границах, очерченных принятыми допущениями и характером проверки, и не должны истолковываться шире. Проверка носит качественный характер: она устанавливает соответствие фактического поведения образца ожидаемому на

наборе сценариев, но не является статистической оценкой надёжности на представительной выборке реального программного обеспечения. Модельные сценарии подобраны и детерминированы — они воспроизводят типовые ситуации недопустимого доступа, но не покрывают всего многообразия путей, которыми обращение к охраняемой области может произойти на практике. Все цели однопоточны, в соответствии с принятым допущением; проверка обращений из параллельных потоков, требующая обработки гонки на шаге восстановления охраны, в испытания настоящего этапа не входит и отнесена к направлениям дальнейшей работы. Сценарий переполнения опирается на позиционно-зависимую сборку цели и заданное расположение полей, обеспечивающие стабильный адрес охраняемого поля; это упрощение, принятое ради воспроизводимости, и в общем случае предпочтительно символическое задание области, обоснованное ранее. Наконец, различие легитимных обращений по контексту инициатора, продемонстрированное в первом сценарии, представляет собой эвристику уровня функции: оно опирается на атрибуцию по адресу инструкции и распознаёт инициатора по принадлежности к разрешённому набору функций, но не противостоит обращению, получившему управление в обход начала разрешённой функции. Перечисленные границы не обесценивают полученных результатов, а определяют область их применимости и очерчивают предмет последующих этапов работы.

4.5 Выводы

В настоящей главе решена третья задача этапа — проведена валидация опытного образца на модельных сценариях доступа к памяти. Испытания выполнены по пяти модельным сценариям в единых условиях, с сопоставлением фактического исхода каждого прогона с заданным заранее ожиданием, а также дополнены первичной оценкой накладных расходов.

Основные результаты главы состоят в следующем:

- 1) подтверждены три свойства методики в пределах принятых допущений: корректность отлова недопустимых обращений — сценариями записи из постороннего модуля, из посторонней функции и вследствие переполнения соседнего буфера; отсутствие ложных срабатываний — сценарием легитимной записи при включённом различении по контексту инициатора и контрольным сценарием

обращения к незащищённой области; базовая атрибуция инициатора — на уровне модуля и на уровне функции;

- 2) получена первичная оценка накладных расходов: стоимость одного перехваченного обращения составляет около 220 мкс и определяется преимущественно переключениями контекста и операциями ptrace, тогда как наблюдение без срабатываний практически бесплатно, а накладные расходы линейны по числу срабатываний, что подтверждает приемлемость избирательной охраны;
- 3) получили опытное подтверждение технические проблемы, выявленные при разработке образца: ложное срабатывание на инициализирующей записи и его устранение различием по контексту инициатора продемонстрированы в сценарии легитимной записи, а необходимость символического задания области и принятое для отдельного сценария упрощение в виде позиционно-зависимой сборки — в сценарии переполнения;
- 4) очерчены границы достоверности результатов: качественный характер проверки, подобранность и детерминированность сценариев, допущение об однопоточности и эвристический характер различения по контексту инициатора, — определяющие область применимости полученных результатов и предмет дальнейшей работы.

Тем самым подтверждено, что методика контекстного мониторинга, разработанная на первом шаге этапа, и реализующий её опытный образец работоспособны на контролируемом наборе сценариев и ведут себя в соответствии с ожиданиями. В совокупности с результатами предыдущих глав это завершает решение задач этапа: осуществлён выбор технологического стека и разработана методика, реализован опытный образец модуля перехвата и проведена его валидация. Полученный образец и подтверждённая на нём методика образуют основу для последующего этапа работы — расширения контроля на чтение и исполнение, перехода к режиму подключения к работающему процессу и проработки многопоточного случая, отнесённых к направлениям дальнейшего развития инструмента.

ЗАКЛЮЧЕНИЕ

В рамках настоящего этапа научно-исследовательской работы решены три поставленные задачи: осуществлён выбор технологического стека и разработана методика контекстного мониторинга доступа к памяти, разработан опытный образец модуля перехвата событий доступа к памяти и проведена его валидация на модельных сценариях. Тем самым выполнен переход от сформированных на предыдущем этапе требований и концептуальной архитектуры к проработанной методике и подтверждённому на практике опытному образцу. Поставленные задачи решены в полном объёме в пределах принятых модельных допущений — однопоточности целевого процесса, режима запуска цели под контролем агента и контроля единственного типа доступа, — принятых сознательно в целях управления рисками и не нарушающих общности подхода.

Основные результаты работы

В качестве уровня реализации обосновано пользовательское пространство на немодифицированном ядре, а базовым механизмом перехвата по итогам сравнения выбрана связка ptrace с mprotect — единственная, совмещающая вынесение наблюдения во внешний по отношению к цели агент, поддержку всех трёх типов доступа и атрибуцию вплоть до инициировавшей нарушение инструкции. На основе выбранного механизма разработана методика контекстного мониторинга и воплощена в опытном образце, центральным инженерным узлом которого является инъекция системного вызова mprotect в адресное пространство цели. Образец проверен на пяти модельных сценариях, исходы которых совпали с ожидаемыми, и дополнен первичной оценкой накладных расходов, подтвердившей приемлемость избирательной охраны. Полученные результаты применимы как основа инструмента точечной контекстной охраны произвольной объявленной области памяти с атрибуцией инициатора недопустимого обращения, занимающего нишу, не покрываемую существующими средствами аудита, секретной памяти, полной проверки памяти и аппаратной защиты.

Перечень основных результатов этапа состоит в следующем:

- 1) обоснован выбор технологического стека — пользовательское пространство на немодифицированном ядре и связка ptrace с mprotect как базовый механизм перехвата;
- 2) разработана методика контекстного мониторинга доступа к памяти с алгоритмом атрибуции адреса к модулю и функции и указанием границ достижимой точности;
- 3) разработан опытный образец модуля перехвата, реализующий внешний перехват записи в охраняемую область и атрибуцию события к модулю и функции, и по результатам его разработки уточнена методика;
- 4) проведена валидация образца на пяти модельных сценариях и получена первичная оценка накладных расходов, подтвердившие три свойства методики и приемлемость избирательной охраны.

Направления дальнейших исследований

Принятые модельные допущения и сознательно суженный объём опытного образца определяют содержание последующих этапов, на которых развитие инструмента осуществляется без смены технологической основы. Основные направления дальнейших исследований состоят в следующем:

- 1) расширение контроля с записи на чтение и исполнение для реализации остальных прикладных сценариев — охраны секрета в памяти и защиты фрагмента кода;
- 2) снятие принятых допущений — переход к режиму подключения к работающему процессу и проработка многопоточной модели целевого процесса;
- 3) повышение устойчивости охраны и развитие средств различения легитимных и недопустимых обращений в сторону более строгих, чем реализованная эвристика;
- 4) оценка применимости инструмента на реальном программном обеспечении и расширение реакции на нарушение от регистрации к активному пресечению.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Президент Российской Федерации. Указ Президента РФ № 400 «О Стратегии национальной безопасности Российской Федерации» [Электронный ресурс]. 2021. URL: <http://publication.pravo.gov.ru/Document/View/0001202107020002>.
2. Президент Российской Федерации. Доктрина информационной безопасности Российской Федерации (утверждена Указом Президента РФ № 646 от 05.12.2016) [Электронный ресурс]. 2016. URL: <http://static.kremlin.ru/media/acts/files/0001201612060002.pdf>.
3. Государственная Дума Российской Федерации. Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации» [Электронный ресурс]. 2006. URL: http://www.consultant.ru/document/cons_doc_LAW_61798/.
4. Monitoring Tools Market (2023–2030): Market Analysis Report: technical report. 2023.
5. Chen H., Zhang W. Memory Analysis for Malware Detection: A Comprehensive Survey // Proceedings of the IEEE Security and Privacy Workshops. 2022. Сс. 112–125.
6. Kumar S., Li P. Obfuscated Memory Malware Detection // IEEE Transactions on Dependable and Secure Computing. 2021. Т. 19, № 5. Сс. 3456–3472.
7. Li C., Lee S.-seob, Zhong L. Blindfold: Confidential Memory Management by Untrusted Operating System // Proceedings of the Network and Distributed System Security Symposium (NDSS). 2025.
8. Chen H., Li P., Zhang W. A Survey of Memory Protection Mechanisms for Operating Systems // ACM Computing Surveys. 2020. Т. 53, № 2. Сс. 1–36.
9. Corbet J. Guard pages for file-backed memory [Электронный ресурс]. 2025. URL: <https://lwn.net/Articles/1011366/>.
10. GrapheneOS. hardened_malloc: Hardened allocator designed for modern systems [Электронный ресурс]. URL: https://github.com/GrapheneOS/hardened_malloc.
11. Kilic O. O. и др. MemGaze: Rapid and Effective Load-Level Memory Trace Analysis // Proceedings of the IEEE International Conference on Cluster Computing (CLUSTER). 2022. Сс. 484–495.
12. Venkataramani G. и др. MemTracker: Efficient and Programmable Support for Memory Access Monitoring and Debugging // Proceedings of the

13th International Symposium on High-Performance Computer Architecture (HPCA). 2007. Сс. 273–284.

13. Arcangeli A., others. Использование userfaultfd для мониторинга доступа к памяти. 2020.
14. Li C., Lee S.-seob, Zhong L. Blindfold: Confidential Memory Management by Untrusted Operating System // Proceedings of the Network and Distributed System Security Symposium (NDSS). 2025.
15. Corbet J. Guard pages for file-backed memory [Электронный ресурс]. 2025. URL: <https://lwn.net/Articles/1011366/>.
16. The Linux kernel development community. Soft-Dirty PTEs — The Linux Kernel documentation [Электронный ресурс]. URL: <https://docs.kernel.org/admin-guide/mm/soft-dirty.html>.
17. The Linux man-pages project. proc_pid_maps(5) — Linux manual page [Электронный ресурс]. URL: https://man7.org/linux/man-pages/man5/proc_pid_maps.5.html.
18. TIS Committee. Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification, Version 1.2. 1995.
19. DWARF Debugging Information Format Committee. DWARF Debugging Information Format, Version 5: technical report. 2017.

ПРИЛОЖЕНИЕ А

Исходный код агента-наблюдателя

```
// Агент-наблюдатель: оркестрация главного цикла мониторинга.
#include "agent.h"
#include "ptrace_util.h"
#include "guard.h"
#include "inject.h"
#include "attrib.h"
#include "event.h"

#include <sys/user.h>
#include <stdio.h>
#include <string.h>
#include <signal.h>

#define MAX_MAPS 1024

// Разрешён ли инициатор: имя функции присутствует в белом списке.
static int func_allowed(const agent_config_t *cfg, const char *func)
{
    if (cfg->allow_count == 0) return 0;    // список пуст – никто не
    разрешён
    if (!func || func[0] == '\\0') return 0;
    for (int i = 0; i < cfg->allow_count; i++)
        if (strcmp(cfg->allow_funcs[i], func) == 0)
            return 1;
    return 0;
}

int agent_run(const agent_config_t *cfg) {
    stop_info_t st;
    int exit_code = 0;

    // 1) запустить цель под трассировкой
    pid_t pid = pt_launch(cfg->target_argv);
    if (pid < 0) return 1;

    // 2) первая остановка – на ехесве целевой программы
    if (pt_wait(pid, &st) < 0) return 1;
    if (st.reason != STOP_SIGNAL) {
        fprintf(stderr, "agent: цель не остановилась на старте\n");
    }
}
```

```

        return 1;
    }

    // 3) найти инструкцию syscall для инъекции mprotect
    uintptr_t syscall_insn = inject_find_syscall_insn(pid);
    if (!syscall_insn) {
        fprintf(stderr, "agent: не найдена инструкция syscall в
цели\n");
        return 1;
    }

    // 4) разрешить охраняемую область
    guard_region_t region;
    if (guard_resolve(pid, &cfg->guard, &region) < 0) {
        fprintf(stderr, "agent: не удалось разрешить охраняемую
область\n");
        return 1;
    }
    fprintf(stderr, "agent: охрана области 0x%lx (%zu байт), страницы
0x%lx..+0x%zx\n",
            (unsigned long)region.addr, region.length,
            (unsigned long)region.page_addr, region.page_span);

    // 5) установить охрану – снять право записи
    if (guard_protect(pid, syscall_insn, &region) < 0) {
        fprintf(stderr, "agent: не удалось установить охрану\n");
        return 1;
    }

    FILE *log = event_log_open(cfg->log_path);
    uint64_t seq = 0;

    // 6) главный цикл
    for (;;) {
        if (pt_continue(pid, 0) < 0) break;
        if (pt_wait(pid, &st) < 0) break;

        if (st.reason == STOP_EXITED) {
            fprintf(stderr, "agent: цель завершилась с кодом %d\n",
st.exit_code);
            exit_code = st.exit_code;
            break;
        }
    }

```

```

        if (st.reason == STOP_SIGNALED) {
            fprintf(stderr, "agent: цель убита сигналом %d\n",
st.signal);
            break;
        }
        if (st.reason != STOP_SIGNAL) break;

        if (st.signal == SIGSEGV) {
            uintptr_t fault = 0;
            if (pt_get_siginfo(pid, &fault) < 0) break;

            int in_object = guard_contains(&region, fault);
            int on_guarded_page = (fault >= region.page_addr &&
                                fault <  region.page_addr +
region.page_span);

            if (in_object || on_guarded_page) {
                // Нарушение на охраняемой странице. Если попало в
сам объект —
                // регистрируем событие; если в соседние данные той
же страницы —
                // это следствие страничной гранулярности mprotect,
событие не
                // регистрируем, но обращение обязаны пропустить
(иначе цель
                // заикнется на нём). В обоих случаях пропускаем
через

                // "снять права -> шаг -> вернуть охрану".
                if (in_object) {
                    struct user_regs_struct regs;
                    if (pt_get_regs(pid, &regs) < 0) break;

                    map_entry_t maps[MAX_MAPS];
                    int n = attrib_read_maps(pid, maps, MAX_MAPS);

                    event_t ev;
                    memset(&ev, 0, sizeof(ev));
                    ev.tid = pid;                                // однопоточная цель:
TID == PID

                    ev.fault_addr = fault;
                    ev.rip = (uintptr_t)regs.rip;
                    ev.access_type = 'W';
                    if (n > 0)

```



```

        attrib_resolve_addr(maps, n, ev.rip,
&ev.attrib);

        // Различение по инициатору: запись из функции
белого списка
        // считается легитимной и не регистрируется как
нарушение.
        if (!func_allowed(cfg, ev.attrib.function)) {
            ev.seq = ++seq;
            event_log_write(log, &ev);
        }
    }

    // пропустить обращение: вернуть права -> один шаг ->
снять снова
    if (guard_unprotect(pid, syscall_insn, &region) < 0)
break;

    if (pt_singlestep(pid, 0) < 0) break;
    if (pt_wait(pid, &st) < 0) break;
    if (st.reason == STOP_EXITED) { exit_code =
st.exit_code; break; }
    if (st.reason == STOP_SIGNALED) break;
    if (guard_protect(pid, syscall_insn, &region) < 0)
break;

    } else {
        // настоящий чужой сегфолт вне охраняемой страницы –
пробросить
        if (pt_continue(pid, SIGSEGV) < 0) break;
        if (pt_wait(pid, &st) < 0) break;
        if (st.reason == STOP_EXITED) { exit_code =
st.exit_code; break; }
        if (st.reason == STOP_SIGNALED) break;
    }
    } else if (st.signal == SIGTRAP) {
        // служебная остановка – продолжаем
        continue;
    } else {
        // прочие сигналы – пробрасываем цели
        if (pt_continue(pid, st.signal) < 0) break;
        if (pt_wait(pid, &st) < 0) break;
        if (st.reason == STOP_EXITED) { exit_code = st.exit_code;
break; }
    }
}

```

```

    }

    event_log_close(log);
    return exit_code;
}

```

Листинг А.1 — Главный цикл наблюдения (agent.c)

```

/*
Инжекция системного вызова mprotect в адресное пространство цели.

Внешний агент не может изменить права чужого адресного пространства
напрямую:
mprotect действует на вызывающий процесс. Поэтому агент заставляет
САМУ цель
исполнить mprotect, временно подменив её регистры и указатель
инструкций.
*/
#include "inject.h"
#include "ptrace_util.h"

#include <sys/user.h>
#include <sys/syscall.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <inttypes.h>

/*
Ищем готовую инструкцию syscall (опкод 0f 05) в исполнимом
отображении цели.
Это позволяет не записывать собственный код в чужую память: мы лишь
указываем rip на уже существующую инструкцию. Сканируем исполнимые
регионы
из /proc/<pid>/maps, читая их через /proc/<pid>/mem.
*/
uintptr_t inject_find_syscall_insn(pid_t pid) {
    char maps_path[64];
    snprintf(maps_path, sizeof(maps_path), "/proc/%d/maps",
(int)pid);
    FILE *f = fopen(maps_path, "r");
    if (!f) { perror("fopen maps"); return 0; }

    char mem_path[64];

```

```

snprintf(mem_path, sizeof(mem_path), "/proc/%d/mem", (int)pid);
FILE *mem = fopen(mem_path, "rb");
if (!mem) { perror("fopen mem"); fclose(f); return 0; }

char line[512];
uintptr_t found = 0;
while (!found && fgets(line, sizeof(line), f)) {
    uintptr_t start, end;
    char perms[5] = {0};
    if (sscanf(line, "%" SCNXPTR "-%" SCNXPTR " %4s", &start,
&end, perms) != 3)
        continue;
    if (perms[2] != 'x')    // нужен исполнимый регион
        continue;

    size_t span = end - start;

    unsigned char *buf = malloc(span);
    if (!buf) break;

    if (fseeko(mem, (off_t)start, SEEK_SET) == 0 &&
        fread(buf, 1, span, mem) >= 2) {
        for (size_t i = 0; i + 1 < span; i++) {
            if (buf[i] == 0x0f && buf[i + 1] == 0x05) {
                found = start + i;
                break;
            }
        }
    }
    free(buf);
}

fclose(mem);
fclose(f);
return found;
}

int inject_mprotect(pid_t pid, uintptr_t syscall_insn,
                    uintptr_t addr, size_t len, int prot) {
    struct user_regs_struct saved, work;
    stop_info_t st;

    if (syscall_insn == 0) {

```

```

        fprintf(stderr, "inject_mprotect: нет адреса инструкции
syscall\n");
        return -1;
    }

    // 1) сохранить регистры цели
    if (pt_get_regs(pid, &saved) < 0) return -1;

    // 2) подготовить регистры для вызова mprotect(addr, len, prot)
    work = saved;
    work.rip = syscall_insn;
    work.rax = __NR_mprotect;           // номер системного вызова
    work.rdi = addr;                    // 1-й аргумент: адрес
    work.rsi = len;                     // 2-й аргумент: длина
    work.rdx = (unsigned long)prot;     // 3-й аргумент: права

    if (pt_set_regs(pid, &work) < 0) return -1;

    // 3) исполнить одну инструкцию – это и будет syscall
    if (pt_singlestep(pid, 0) < 0) return -1;
    if (pt_wait(pid, &st) < 0) return -1;
    if (st.reason != STOP_SIGNAL) {
        fprintf(stderr, "inject_mprotect: неожиданная остановка после
syscall\n");
        return -1;
    }

    // 4) прочитать результат mprotect из rax
    if (pt_get_regs(pid, &work) < 0) return -1;
    long ret = (long)work.rax;

    // 5) восстановить исходные регистры цели
    if (pt_set_regs(pid, &saved) < 0) return -1;

    if (ret != 0) {
        fprintf(stderr, "inject_mprotect: mprotect вернул %ld\n",
ret);
        return -1;
    }
    return 0;
}

```

Листинг A.2 — Инжекция системного вызова mprotect (inject.c)

```

// Установка/снятие охраны над объявленной областью и разрешение её
адреса.
#include "guard.h"
#include "inject.h"
#include "attrib.h"

#include <sys/mman.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#define MAX_MAPS 1024

static size_t page_size_cached(void) {
    long ps = sysconf(_SC_PAGESIZE);
    return ps > 0 ? (size_t)ps : 4096;
}

int guard_resolve(pid_t pid, const guard_spec_t *spec, guard_region_t
*region) {
    memset(region, 0, sizeof(*region));

    uintptr_t addr = 0;
    size_t sym_size = 0;
    if (spec->symbol_name) {
        map_entry_t maps[MAX_MAPS];
        int n = attrib_read_maps(pid, maps, MAX_MAPS);
        if (n <= 0) return -1;
        addr = attrib_resolve_symbol(maps, n, spec->symbol_name,
&sym_size);
        if (!addr) {
            fprintf(stderr, "guard_resolve: символ '%s' не найден\n",
spec->symbol_name);
            return -1;
        }
    } else if (spec->abs_addr) {
        addr = spec->abs_addr;
    } else {
        fprintf(stderr, "guard_resolve: не задан ни символ, ни
адрес\n");
        return -1;
    }
}

```

```

    size_t ps = page_size_cached();
    region->addr = addr;
    // длина области: явно заданная (--len) важнее всего; иначе
размер символа
    // из таблицы ELF; иначе дефолт 8 байт.
    if (spec->length)
        region->length = spec->length;
    else if (sym_size)
        region->length = sym_size;
    else
        region->length = 8;
    region->page_addr = addr & ~(ps - 1);
    // сколько байт страниц покрывает область [addr, addr+length)
    uintptr_t last = addr + region->length - 1;
    uintptr_t last_page = last & ~(ps - 1);
    region->page_span = (last_page - region->page_addr) + ps;
    return 0;
}

int guard_protect(pid_t pid, uintptr_t syscall_insn, const
guard_region_t *region) {
    // снять право записи: страницы области -> только чтение
    return inject_mprotect(pid, syscall_insn,
                           region->page_addr, region->page_span,
PROT_READ);
}

int guard_unprotect(pid_t pid, uintptr_t syscall_insn, const
guard_region_t *region) {
    // временно вернуть запись для одиночного шага
    return inject_mprotect(pid, syscall_insn,
                           region->page_addr, region->page_span,
PROT_READ | PROT_WRITE);
}

int guard_contains(const guard_region_t *region, uintptr_t
fault_addr) {
    if (!region) return 0;
    return fault_addr >= region->addr &&
           fault_addr < region->addr + region->length;
}

```

Листинг А.3 — Установка и восстановление охраны области (guard.c)

```
/*
```

Атрибуция: привязка адреса инструкции-инициатора к модулю и функции.
Реализует алгоритм методики: /proc/<pid>/maps -> модуль+смещение -> ELF-символ.

```
*/
```

```
#include "attrib.h"
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <inttypes.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <elf.h>
```

```
int attrib_read_maps(pid_t pid, map_entry_t *out, int max) {
```

```
    char path[64];
```

```
    snprintf(path, sizeof(path), "/proc/%d/maps", (int)pid);
```

```
    FILE *f = fopen(path, "r");
```

```
    if (!f) { perror("fopen maps"); return -1; }
```

```
    char line[512];
```

```
    int n = 0;
```

```
    while (n < max && fgets(line, sizeof(line), f)) {
```

```
        map_entry_t e;
```

```
        memset(&e, 0, sizeof(e));
```

```
        char perms[5] = {0};
```

```
        int consumed = 0;
```

```
        // start-end perms offset dev inode path
```

```
        int got = sscanf(line, "%" SCNXPTR "-%" SCNXPTR " %4s %" SCNXPTR " %*s %*s %n",
```

```
                        &e.start, &e.end, perms, &e.file_offset, &consumed);
```

```
        if (got < 4) continue;
```

```
        memcpy(e.perms, perms, 4);
```

```
        // путь модуля – остаток строки после inode (если есть)
```

```
        if (consumed > 0) {
```

```
            char *p = line + consumed;
```

```
            char *nl = strchr(p, '\n');
```

```
            if (nl) *nl = '\0';
```

```
            while (*p == ' ') p++;
```

```
            strncpy(e.path, p, sizeof(e.path) - 1);
```

```

    }
    out[n++] = e;
}
fclose(f);
return n;
}

```

```

const map_entry_t *attrib_find_map(const map_entry_t *maps, int n,
uintptr_t addr) {
    for (int i = 0; i < n; i++)
        if (addr >= maps[i].start && addr < maps[i].end)
            return &maps[i];
    return NULL;
}

```

/*

Базовый адрес загрузки модуля: минимальный start среди всех отображений с тем же путём. Нужен, чтобы перевести виртуальный адрес в смещение, не зависящее от ASLR, и обратно.

*/

```

static uintptr_t module_base(const map_entry_t *maps, int n, const
char *path) {
    uintptr_t base = (uintptr_t)-1;
    for (int i = 0; i < n; i++)
        if (maps[i].path[0] && strcmp(maps[i].path, path) == 0)
            if (maps[i].start < base) base = maps[i].start;
    return base == (uintptr_t)-1 ? 0 : base;
}

```

/*

Читаем ELF-файл модуля и ищем функцию по mod_off – смещению внутри модуля

(rip - base). st_value символов приводим к тому же смещению:

ET_EXEC: st_value – абсолютный VA, смещение = st_value - base;

ET_DYN : st_value уже является смещением от базы.

*/

```

static int elf_lookup_symbol_by_offset(const char *module_path,
uint64_t mod_off,
                                int et_dyn, uintptr_t base,
                                char *out_name, size_t
out_sz) {

```



```

    int fd = open(module_path, O_RDONLY);
    if (fd < 0) return 0;

    int ok = 0;
    Elf64_Ehdr eh;
    if (read(fd, &eh, sizeof(eh)) != (ssize_t)sizeof(eh))
{ close(fd); return 0; }
    if (memcmp(eh.e_ident, ELF_MAGIC, SELFMAGIC) != 0) { close(fd); return
0; }

    Elf64_Shdr *sh = calloc(eh.e_shnum, sizeof(Elf64_Shdr));
    if (!sh) { close(fd); return 0; }
    if (pread(fd, sh, eh.e_shnum * sizeof(Elf64_Shdr), eh.e_shoff) !=
        (ssize_t)(eh.e_shnum * sizeof(Elf64_Shdr))) { free(sh);
close(fd); return 0; }

    for (int i = 0; i < eh.e_shnum && !ok; i++) {
        if (sh[i].sh_type != SHT_SYMTAB && sh[i].sh_type !=
SHT_DYNSYM)
            continue;
        Elf64_Shdr *strsh = &sh[sh[i].sh_link];

        char *symtab = malloc(sh[i].sh_size);
        char *strtab = malloc(strsh->sh_size);
        if (!symtab || !strtab) { free(symtab); free(strtab);
continue; }

        if (pread(fd, symtab, sh[i].sh_size, sh[i].sh_offset) ==
(ssize_t)sh[i].sh_size &&
            pread(fd, strtab, strsh->sh_size, strsh->sh_offset) ==
(ssize_t)strsh->sh_size) {

            int cnt = sh[i].sh_size / sizeof(Elf64_Sym);
            Elf64_Sym *syms = (Elf64_Sym *)symtab;
            for (int s = 0; s < cnt; s++) {
                if (ELF64_ST_TYPE(syms[s].st_info) != STT_FUNC)
continue;

                if (syms[s].st_value == 0 || syms[s].st_size == 0)
continue;

                // смещение символа от базы модуля
                uint64_t sym_off = et_dyn ? syms[s].st_value
                    : (syms[s].st_value -
base);

```

```

        if (mod_off >= sym_off && mod_off < sym_off +
syms[s].st_size) {
            const char *nm = strtab + syms[s].st_name;
            strncpy(out_name, nm, out_sz - 1);
            out_name[out_sz - 1] = '\\0';
            ok = 1;
            break;
        }
    }
    free(symtab);
    free(strtab);
}

free(sh);
close(fd);
return ok;
}

```

```

uintptr_t attrib_resolve_symbol(const map_entry_t *maps, int n, const
char *symbol,

```

```

                                size_t *out_size) {
    // Ищем символ по имени во всех модулях с непустым путём.
    Возвращаем
    // фактический виртуальный адрес в адресном пространстве цели.
    if (out_size) *out_size = 0;
    char seen[64][ATTRIB_NAME_MAX];
    int seen_n = 0;

    for (int i = 0; i < n; i++) {
        if (!maps[i].path[0]) continue;
        if (maps[i].path[0] == '[') continue;    // [heap], [stack] и
т.п.

        // пропускаем уже обработанные модули
        int dup = 0;
        for (int k = 0; k < seen_n; k++)
            if (strcmp(seen[k], maps[i].path) == 0) { dup = 1;
break; }
        if (dup) continue;
        if (seen_n < 64) strncpy(seen[seen_n++], maps[i].path,
ATTRIB_NAME_MAX - 1);

```

```

int fd = open(maps[i].path, 0_RDONLY);
if (fd < 0) continue;

Elf64_Ehdr eh;
if (read(fd, &eh, sizeof(eh)) != (ssize_t)sizeof(eh) ||
    memcmp(eh.e_ident, ELF_MAGIC, SELF_MAGIC) != 0) { close(fd);
continue; }
int et_dyn = (eh.e_type == ET_DYN);
uintptr_t base = module_base(maps, n, maps[i].path);

Elf64_Shdr *sh = calloc(eh.e_shnum, sizeof(Elf64_Shdr));
if (!sh) { close(fd); continue; }
if (pread(fd, sh, eh.e_shnum * sizeof(Elf64_Shdr),
eh.e_shoff) !=
    (ssize_t)(eh.e_shnum * sizeof(Elf64_Shdr))) { free(sh);
close(fd); continue; }

uintptr_t result = 0;
for (int s_i = 0; s_i < eh.e_shnum && !result; s_i++) {
    if (sh[s_i].sh_type != SHT_SYMTAB && sh[s_i].sh_type !=
SHT_DYNSYM)
        continue;
    Elf64_Shdr *strsh = &sh[sh[s_i].sh_link];
    char *symtab = malloc(sh[s_i].sh_size);
    char *strtab = malloc(strsh->sh_size);
    if (!symtab || !strtab) { free(symtab); free(strtab);
continue; }

    if (pread(fd, symtab, sh[s_i].sh_size, sh[s_i].sh_offset)
== (ssize_t)sh[s_i].sh_size &&
        pread(fd, strtab, strsh->sh_size, strsh->sh_offset)
== (ssize_t)strsh->sh_size) {
        int cnt = sh[s_i].sh_size / sizeof(Elf64_Sym);
        Elf64_Sym *syms = (Elf64_Sym *)symtab;
        for (int s = 0; s < cnt; s++) {
            if (syms[s].st_name == 0) continue;
            const char *nm = strtab + syms[s].st_name;
            if (strcmp(nm, symbol) == 0 && syms[s].st_value !=
0) {
                result = et_dyn ? (base + syms[s].st_value)
                :
                (uintptr_t)syms[s].st_value;
                if (out_size) *out_size =

```

```

(size_t)syms[s].st_size;
                                break;
                                }
                                }
                                }
                                free(symtab);
                                free(strtab);
                                }
                                free(sh);
                                close(fd);
                                if (result) return result;
                                }
                                return 0;
                                }

int attrib_resolve_addr(const map_entry_t *maps, int n, uintptr_t
rip,
                                attrib_result_t *res) {
    memset(res, 0, sizeof(*res));
    const map_entry_t *m = attrib_find_map(maps, n, rip);
    if (!m) return -1;

    // модуль и смещение
    const char *path = m->path[0] ? m->path : "[anonymous]";
    strncpy(res->module, path, sizeof(res->module) - 1);
    uintptr_t base = m->path[0] ? module_base(maps, n, m->path) : m-
>start;
    res->module_offset = rip - base;

    // попытка разрешить функцию по ELF
    if (m->path[0] && m->path[0] != '[') {
        int fd = open(m->path, O_RDONLY);
        if (fd >= 0) {
            Elf64_Ehdr eh;
            if (read(fd, &eh, sizeof(eh)) == (ssize_t)sizeof(eh) &&
                memcmp(eh.e_ident, ELFMAG, SELFMAG) == 0) {
                int et_dyn = (eh.e_type == ET_DYN);
                // всегда ищем по смещению инструкции внутри модуля
                char fname[ATTRIB_NAME_MAX] = {0};
                if (elf_lookup_symbol_by_offset(m->path, res-
>module_offset,
                                                et_dyn, base,
                                                fname,

```

```

sizeof(fname))) {
    strncpy(res->function, fname, sizeof(res-
>function) - 1);
    res->has_function = 1;
}
}
close(fd);
}
}
return 0;
}

```

Листинг A.4 — Атрибуция события к модулю и функции (attrib.c)